DTU Compute
Department of Applied Mathematics and Computer Science

Agents with Small Language Models

Evaluating Optimization Techniques for Reliable Tool Calling

Master Thesis

Paulo Ricardo Beckhauser de Araujo

Agents with Small Language Models
Evaluating Optimization Techniques for Reliable Tool Calling

Master Thesis
July, 2026

By
Paulo Ricardo Beckhauser de Araujo

Cover photo: Vibeke Hempler, 2012
Published by: DTU, Department of Applied Mathematics and Computer Science, Richard Petersens Plads, Building 324, 2800 Kgs. Lyngby Denmark
www.compute.dtu.dk
ISSN: [0000-0000] (electronic version)
ISBN: [000-00-0000-000-0] (electronic version)

ISSN: [0000-0000] (printed version)
ISBN: [000-00-0000-000-0] (printed version)

Approval

This Master's thesis was prepared in fulfillment of the requirements for the degree Master of Science in Engineering, MSc Autonomous Systems at the Technical University of Denmark (DTU). The thesis was prepared by Paulo Ricardo Beckhauser de Araujo (student ID: s242779) under supervision of Nicki Skafte Detlefsen, Professor at DTU Compute. The project period lasted 6 months from January 5th, 2026 until July 5th, 2026, equal to a workload of 35 ECTS points.

The reader is expected to be familiar with the basic concepts of linear algebra, programming languages, machine learning, deep learning and generative AI.

Paulo Ricardo Beckhauser de Araujo - s242779

.....

Signature

.....

Date

Abstract

Frontier language models are now the default backbone for agentic systems, but their dependence on external APIs limits their use in offline, privacy-sensitive, and high-volume deployments. Small language models can run on consumer hardware and avoid these constraints. However, they often fail at the structured output and tool-calling tasks that agentic pipelines require. This thesis studies how far optimization techniques can move a small model toward reliable tool calling, and what accuracy cost or benefit each technique introduces.

The study uses an incremental ablation over the Qwen 2.5 model family (0.5B–7B), with `Qwen2.5-7B-Instruct` as the primary model. The models are evaluated on four BFCL v4 categories (`simple_python`, `multiple`, `parallel`, and `parallel_multiple`) and on τ-bench retail. A minimal model-agnostic baseline that does not use the model's native tool-calling template scores 1.5% on `simple_python` under a strict whole-completion parser. This is 77–96 percentage points below frontier models on the same category (78.75%–97.75%). The same 7B run scores 62.00% when a lenient parser accepts the first complete JSON object and ignores trailing prose, while a control configuration using the model's native template reaches 96.00%, inside the frontier band. The unoptimized gap therefore measures the cost of a generic integration and strict parse contract, not the model's capability ceiling.

Constrained decoding is the dominant intervention. It raises accuracy to 72.75% without modifying model weights or generating additional tokens, while guaranteeing parseable structured output and reducing mean per-call latency from 6.995 s under unconstrained generation to 0.878 s. Relative to the lenient 7B parser baseline, this is a 10.75 percentage point accuracy gain. Relative to the strict generic floor, it is a 71.25 percentage point recovery. AWQ INT4 quantization preserves most of this accuracy at 72.25% while reducing memory from 14.25 GiB to 5.20 GiB. In contrast, few-shot prompting and chain-of-thought reasoning reduce accuracy by 2.5 and 6.75 percentage points relative to the constrained-decoding configuration each extends. Retrieval-augmented generation reduces accuracy by 24.5 percentage points in an open-catalogue setting that replaces the single oracle candidate with five retrieved candidates. The drop reflects candidate disambiguation (retrieval recall@5 is 97.2%), not retrieval overhead on a fixed task. Format-aligned LoRA fine-tuning reaches 76.75%, and 74.25% after subsequent quantization at the same memory footprint.

The strongest no-training result is schema-enriched constrained decoding (CD+schema), which adds parameter descriptions, enumeration values, and default values to the argument extraction prompt while keeping the FSM structural constraint unchanged. CD+schema reaches 89.00% (+16.25 pp over plain CD, McNemar $p < 0.00001$), placing the 7B model inside the frontier range on the single-call, single-candidate BFCL category without modifying model weights. These results show that the unoptimized failure mode is primarily a format-compliance gap: 394 of 400 baseline failures are malformed outputs rather than wrong answers. Constrained decoding removes that gap structurally, and schema enrichment recovers much of the remaining loss by restoring schema information discarded by the generic prompt. The result does not imply end-to-end agent reliability. Accuracy falls to 4.35% on τ-bench retail, and BFCL `parallel` scores 0% under the initial single-call schema, though extending the schema to joint multi-call generation lifts it to 79% at 7B. The persistent τ-bench gap shows that multi-turn planning, unlike multi-call emission, requires more than the optimization techniques studied here.

Acknowledgements

This thesis was completed with the guidance and support of **Nicki Skafte Detlefsen**, Associate Professor at the Technical University of Denmark, who supervised this work.

I would also like to thank the professors and friends who were part of my journey throughout my master's degree, and above all, my family for their unwavering support along the way.

Contents

1 Introduction

1.1 Motivation

Large language models (LLMs) have become the dominant backbone for agentic systems. These systems reason over user requests, decompose tasks, and call external tools to produce structured outputs. Frontier models such as GPT and Claude perform strongly on these tasks. However, they also introduce high inference costs, external API dependencies, added latency, and limited control over data privacy. In many deployment settings, these constraints determine whether the system can be used at all.

The dominant constraint depends on the setting. In on-device and edge applications, such as a phone assistant or an in-vehicle agent, there may be no reliable network path to an external API. Inference must also fit within a fixed memory and power budget. In regulated domains such as healthcare, legal services, and finance, transmitting user data to a third-party API may be prohibited regardless of cost. In high-throughput backend automation, where a single task can issue many tool calls and request volume can reach millions per day, metered API calls dominate the operating budget.

These settings share a useful structure: many individual tool calls are mechanically simple. The model must emit one valid tool call with correct arguments given a tool schema and the current state. This is the schema- and API-constrained regime where SLMs are most competitive [25]. The practical question behind this thesis is therefore whether a small local model can handle enough of these calls reliably, leaving only the hard cases for a frontier model.

Small language models (SLMs), commonly taken to span roughly 1 to 12 billion parameters [25], offer a possible answer. Recent open-weight models such as Qwen 2.5 [24], Llama 3.2 [20], and Phi-4 [1] can run on consumer-grade hardware, support local deployment, and sharply reduce per-query costs. Out of the box, however, SLMs still struggle with capabilities that agentic systems require. They can produce malformed structured outputs, select incorrect tools, hallucinate function arguments, and fail at multi-step reasoning. These shortcomings have led to a widespread assumption that reliable agentic behavior requires large-scale models.

The failure mode is concrete, and the aggregate figures that follow are measured on BFCL v4 `simple_python`. Consider a user query such as *"What is the sum of 265 and 345?"* directed at Qwen 2.5 7B-Instruct without any optimization, where the available tool is an `add` function that accepts two numeric arguments. The expected final output of the evaluation pipeline is a single valid function call, `{"name": "add", "arguments": {"a": 265, "b": 345}}`, assembled by selecting the function name in the first stage and extracting its arguments in the second. A logged generation from the argument-extraction stage (temperature 0) instead produces the following, reproduced verbatim:

````
{"a": 265, "b": 345} The function ‘add‘ takes two numbers as
arguments and returns their sum as a string. [...] To compute the
sum, you would call the function like this:

‘‘‘javascript
add(265, 345)
‘‘‘
````

```
The result would be:

‘‘‘json
"610"
‘‘‘

So, the sum of 265 and 345 is 610.
```

The model has identified the correct arguments—the leading object `{"a": 265, "b": 345}` is itself well-formed—and the correct tool, named in the explanatory text that follows, but that trailing content makes the completion fail to parse as a single JSON object (a Python `json Extra data` error at the first character past the object). On BFCL v4 `simple_python`, 394 out of 400 responses from the unoptimized 7B model fail in this way, yielding 1.5% overall accuracy (6/400). The problem is not that the model lacks the knowledge to answer; it is that the model cannot reliably express that knowledge in the structured format that agentic systems require.

This 1.5% figure is the floor of a deliberately minimal, model-agnostic integration, not the model's ceiling. It reflects plain-text prompts, a strict single-object parse that discards the leading valid object once trailing text follows it, and no use of the model-specific tool-calling template that Qwen 2.5 provides. A more lenient parser that accepts the leading valid object and ignores any trailing text recovers many of these completions: the same 7B run then scores 62.00% (this lenient parser is the one used for the isolation analysis in Chapter 4). A control configuration that prompts the same model through its native tool-calling template (B-template) reaches 96.00% (384/400) on the same test cases. The 1.5% should therefore be read as the floor for a generic integration rather than an intrinsic limit, and the contribution of constrained decoding examined here is that it restores format compliance structurally, with guarantees that hold for any model and any schema. The full parser-sensitivity and B-template analysis is deferred to Chapters 4 and 5.

This thesis investigates one part of that assumption: whether reliable single-call tool use requires a large model, or whether much of the observed failure comes from insufficient structure around generation. The evaluation uses four Qwen 2.5 Instruct sizes (0.5B, 1.5B, 3B, and 7B) on BFCL v4 `simple_python`, with a main ladder of optimization configurations and additional isolation variants. It asks how much of the failure rate is caused by output format, how much remains as semantic argument error, and which interventions improve each part.

The main result is that constrained decoding removes format-compliance failures at every model size and makes every output parseable under the required schema. Against the strict generic floor, accuracy rises from 1.5–4.75% in the unoptimized setting to 51.50–72.75%, depending on size, without modifying model weights. At 7B, where the same unguided run is also evaluated under the lenient first-object parser, the same-parser gain is smaller but still positive: 62.00% to 72.75% (+10.75 pp). The main contribution of constrained decoding is therefore the model-agnostic structural guarantee, with an accuracy gain whose apparent size depends on the parse contract. AWQ INT4 quantization reduces memory by 63.5% at the 7B size (from 14.25 GiB to 5.20 GiB) at a cost of at most 0.5 pp above 1.5B, within run-to-run variance. Format-aligned LoRA fine-tuning improves accuracy at every size; at 7B, CD+FT-aligned reaches 76.75%, within single-run uncertainty of the weakest frontier entry on the same category (GPT-5-mini, 78.75%).

The strongest result is schema-enriched constrained decoding (Config CD+schema). This

configuration adds parameter descriptions, enumeration values, and defaults to the argument extraction prompt while keeping the FSM mask unchanged. At 7B it reaches 89.00% (+16.25 pp over plain CD, McNemar's test $p < 0.00001$). This places the model in the lower frontier range on the single-call, single-candidate BFCL v4 Python Simple AST category, above GPT-5-mini, without modifying model weights. The comparison is intentionally narrow: it does not mean that the constrained pipeline is the best Qwen integration, since the native-template control reaches 96.00%, and it does not imply general agent reliability. The failures that persist across configurations and sizes are semantic rather than structural: wrong optional parameter defaults, numeric precision mismatches, and string format conventions that the model's learned priors do not match. This distinction, between failures that constraints can fix and failures that require better semantic guidance or training, organises the evaluation.

The practical deployment context is a cascade architecture: a two-tier system in which a locally deployed small model handles routine requests, while a frontier model accessed through an external API handles the remainder. This pattern is economically motivated. Frontier model APIs are charged per token, whereas a locally deployed 7B model on a single consumer GPU incurs a fixed hardware cost independent of query volume. If the small model handles 75% of requests correctly and a router can identify the remaining cases, 75% of per-query API calls can be avoided.

The central challenge is to increase the fraction of requests that the small model can handle reliably and then route the residual semantic failures. This thesis evaluates that local SLM tier in isolation for tool-calling tasks; it does not implement the cascade router or escalation mechanism. BFCL accuracy is therefore used as an upper-bound proxy for the fraction of single-call requests the SLM tier could handle. It provides a comparable basis for estimating cascade economics across configurations, not a measured deployment saving.

The optimization techniques are ordered by implementation cost. Methods that require no modification to the model weights (constrained decoding, prompt engineering, retrieval-augmented generation, and post-training quantization) are evaluated first, as they can be applied to any pre-trained checkpoint without additional compute. LoRA fine-tuning is evaluated last, as it requires a training run on a GPU cluster. This ordering reflects a practical decision ladder: a deployment team can adopt the no-training configurations immediately and defer the training investment until the marginal accuracy gain justifies it.

1.2 Problem Statement

Agentic systems built on language models need two capabilities: they must reason correctly about user requests, and they must call the right tools with valid arguments. Reasoning covers task decomposition, the decision to call a tool or answer directly, and multi-step logic over intermediate results. Tool calling covers function selection, structured argument generation, and appropriate handling of the tool result. This thesis evaluates tool calling directly. Reasoning is measured only where it appears inside tool selection, argument construction, and one multi-step agentic benchmark; general-purpose reasoning benchmarks are outside the scope of the evaluation.

Without optimization, small language models fail at these tasks in several ways. They generate invalid JSON, select non-existent tools, produce arguments with incorrect types or values, and fail to follow execution plans. These failures are not merely inconvenient. In a production agent, they can prevent the system from calling tools at all.

Two categories of failure are distinguishable. Format failures occur when the model produces syntactically invalid output: malformed JSON, unclosed brackets, incorrect field names,

or free-form text where a structured object is required. These failures are independent of whether the model understood the query; they reflect an inability to express a correct answer in the required format. Semantic failures occur when the output is structurally valid but contains incorrect values: wrong function arguments, hallucinated parameters, or mismatched types. These failures reflect gaps in the model's learned associations between query context and argument conventions.

The distinction matters because the two failure types respond to different interventions. Format failures can be eliminated at inference time by constrained decoding, which masks invalid tokens at each generation step without modifying the model. Semantic failures cannot be addressed by structural constraints; they require either training-based interventions that update the model's learned priors, or retrieval-based methods that ground generation in retrieved factual content. The central question is therefore not whether SLMs fail, but which failure type dominates and which intervention addresses it most effectively. The central finding of this thesis is that constrained decoding converts the problem from format compliance (fixable structurally at inference time) into semantic argument-value accuracy, which only format-aligned training can further improve.

The appropriate intervention depends on the source of failure. If failures come mainly from limited model capacity, then larger models or additional training are required. If they come mainly from insufficient constraints and missing schema guidance, inference-time structure may close much of the gap. This thesis therefore evaluates which part of the failure can be removed structurally and which part remains as semantic error.

1.3 Research Questions

This thesis addresses three research questions:

RQ1 *What is the performance gap between unoptimized small language models (0.5B–7B parameters) and frontier models on tool-calling benchmarks?*

RQ2 *What is the marginal contribution of each optimization technique (constrained decoding, schema enrichment, prompt engineering, chain-of-thought prompting, retrieval-augmented generation, quantization, and LoRA fine-tuning) when applied cumulatively to small models for tool calling?*

RQ3 *Can an optimized small language model configuration provide an upper-bound local tier for cascade-style single-call tool calling on consumer hardware?*

RQ1 establishes the baseline gap that motivates the work. RQ2 is the core empirical contribution. It is answered through an ablation study with two parts: a cumulative ladder that adds one technique at a time, and a complementary isolation arm that evaluates inference-time techniques without constrained decoding. This separates each technique's standalone effect from the contribution of constrained decoding itself. RQ3 addresses the narrower deployment question of whether the optimized configuration can act as a local single-call tier in a cascade-style architecture.

For RQ3, the local-tier criterion is assessed against two explicit conditions. The first is lower-tier frontier parity on the same benchmark category: single-call accuracy within the range spanned by deployed frontier models on BFCL v4 Python Simple AST (78.75%–97.75%; the comparison set is given in Chapter 4). The second is local deployability: a memory footprint that fits a single consumer GPU (24 GiB), combined with single-call accuracy high enough that the local tier could handle a substantial fraction of traffic under an ideal semantic router ($p \geq 0.70$ is used as the reference point in the cost model of Chapter 5). Both criteria are scoped to single-call, schema-constrained tool calling. Neither

covers general production agents or multi-turn task completion, and the routing mechanism required to escalate residual semantic failures is not implemented.

To answer these questions, the thesis evaluates an incremental pipeline of methods ordered by implementation cost. No-training methods are evaluated first: constrained decoding [31], prompt engineering [3], inference-time compute [30], retrieval-augmented generation [17], and quantization [18]. Training-based methods, represented by LoRA fine-tuning [12], are evaluated after the no-training methods. Each method is assessed both individually and in combination to measure its marginal contribution to tool-calling accuracy.

1.4 Contributions

The main contributions of this thesis are:

1. A demonstration that constrained decoding alone gives a model-agnostic structural guarantee for BFCL tool-call generation, lifting the strict model-agnostic floor from 1.5–4.75% to 51.50–72.75% across the Qwen 2.5 size range (0.5B to 7B) without modifying model weights or relying on model-specific prompt templates. At 7B, where the same unguided run is also scored with the lenient first-object parser, constrained decoding improves accuracy from 62.00% to 72.75% (+10.75 pp) while making every output parseable.

2. An incremental ablation study that isolates the marginal contribution of constrained decoding, prompt engineering, inference-time chain-of-thought, retrieval-augmented generation, quantization, LoRA fine-tuning, and schema-enriched prompting to BFCL `simple_python` accuracy. Schema-enriched constrained decoding (CD+schema, 89.00%) is the strongest no-training configuration and the first to enter the lower frontier range on the single-call, single-candidate BFCL v4 Python Simple AST category (10.25 pp above GPT-5-mini, 78.75%); CD+FT-aligned (76.75%) approaches the same lower frontier tier.

3. A scaling analysis across Qwen 2.5 0.5B, 1.5B, 3B, and 7B showing that the strict-parser recovery from constrained decoding scales with model size (48 pp at 0.5B to 71.25 pp at 7B), that the same-parser 7B gain over the lenient unguided baseline is 10.75 pp, that quantization cost is negligible above 1.5B, that few-shot prompting reverses direction at 7B, and that format-aligned fine-tuning improves accuracy at every size.

4. A characterization of the pipeline boundary and the schema change that crosses it: the two-stage single-call architecture achieves 70.5% on the `multiple` category at 7B but 0% on `parallel` by construction, and replacing the per-function extraction stage with a joint multi-call schema lifts `parallel` to 79% at 7B, showing the limit was the output schema rather than model capability.

5. An exploratory real-world validation on the GitHub MCP server (21 live tools, 50 natural-language prompts) showing that constrained decoding holds 96.0% tool selection at 7B when the model must choose from a full catalogue of semantically similar operations, that its benefit concentrates on the weakest viable model, and that forcing a choice over a large catalogue imposes a disambiguation cost on smaller models.

1.5 Thesis Structure

The remainder of this thesis is organized as follows:

Chapter 2 provides the theoretical background on language models, agentic systems, and the optimization techniques investigated in this work, including constrained decoding, prompt engineering, inference-time compute, retrieval-augmented generation, quantization, and LoRA fine-tuning.

Chapter 3 describes the experimental design, the cumulative evaluation ladder, the candidate models, and the evaluation framework including benchmarks and metrics.

Chapter 4 presents the experimental results for each optimization method, both individually and in combination, along with comparisons to frontier models.

Chapter 5 discusses the implications of the results, analyzes the cost-benefit trade-offs of each method, and addresses limitations of the study.

Chapter 6 answers the research question, summarizes the findings, and outlines directions for future work.

2 Background

Reliable tool calling with small language models draws on several areas of prior work. This chapter introduces the language-model architecture behind the evaluated systems, the design of agentic systems that invoke external tools, and the inference-time and training-time techniques used to control model output.

2.1 Language Models

2.1.1 Transformer Architecture

The Transformer architecture, introduced by Vaswani et al. [28], forms the foundation of modern language models. It replaces recurrent computation with self-attention, which allows each token in a sequence to attend to all other tokens in parallel. A Transformer block consists of a multi-head self-attention layer followed by a position-wise feed-forward network, with layer normalization and residual connections around each sub-layer.

Given an input sequence of token embeddings X, the self-attention mechanism computes queries $Q = XW^Q$, keys $K = XW^K$, and values $V = XW^V$ through learned linear projections. The attention output is computed as:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V \tag{2.1}$$

where d_k is the dimension of the key vectors. Multi-head attention runs this operation in parallel across multiple subspaces, allowing the model to capture different types of dependencies simultaneously.

For language modeling, the decoder-only variant is the most common choice [3], and has been shown to achieve the strongest zero-shot generalization among architectures trained with unsupervised pretraining [29]. It applies a causal attention mask so that each token can only attend to preceding tokens, enabling autoregressive generation where the model predicts one token at a time conditioned on all previous tokens.

Efficient autoregressive generation is supported by the key-value (KV) cache, which stores the key and value matrices computed for all previous tokens so that they do not need to be recomputed at each generation step. Without caching, generating a sequence of T tokens requires $O(T^2)$ attention operations; with caching, each new token requires only $O(T)$ operations. The KV cache grows linearly with sequence length and the number of attention heads, and its memory footprint becomes a significant constraint at long context lengths or large batch sizes.

Grouped-query attention (GQA) reduces KV cache memory by sharing a single set of key and value heads across multiple query heads. In standard multi-head attention, each of the h query heads has a corresponding key and value head, resulting in $2h$ matrices stored in the KV cache per layer. With GQA, h query heads are divided into g groups, each sharing one key and one value head, reducing the cache to $2g$ matrices per layer where $g \ll h$. GQA is used in the Qwen 2.5 model family [24] and is a key reason why the 7B model fits within consumer GPU memory budgets during inference.

2.1.2 Scaling Laws

Scaling laws describe how model performance changes with the number of parameters, the amount of training data, and the available compute budget. Kaplan et al. [15] showed

that language model loss follows a power-law relationship with these factors, and that performance improves predictably as they increase.

Hoffmann et al. [11] refined these findings with the Chinchilla scaling laws, showing that many large models were under-trained relative to their size. For a fixed compute budget, performance improves when model size and training data are balanced rather than when parameter count is maximized alone. This matters for small language models because a well-trained small model can outperform a larger but under-trained one.

More recently, the focus has shifted toward inference-time scaling [26], where additional computation at generation time (through techniques such as chain-of-thought prompting or search) can improve performance without increasing model size.

2.1.3 Small vs. Large Language Models

Large language models (LLMs) such as GPT-4 [21] and the Claude model family operate at hundreds of billions of parameters and perform strongly across a wide range of tasks, including complex reasoning and tool use. Their size also makes them expensive to run, typically requiring multi-GPU clusters or cloud API access for inference.

Small language models (SLMs), commonly taken to span roughly 1 to 12 billion parameters [25], can run on consumer hardware such as a single GPU with 24GB of VRAM. Recent open-weight SLMs, including Qwen 2.5 [24], Llama 3.2 [20], and Phi-4 [1], have narrowed the gap with larger models on standard benchmarks. This improvement comes from better training data, improved tokenizers, and architectural refinements such as grouped-query attention (GQA).

Despite these improvements, SLMs still lag behind frontier models on tasks that require multi-step reasoning, tool selection from large schemas, and reliable structured output generation. This gap motivates the optimization techniques explored in this thesis.

2.2 Agentic Systems

An agentic system is a software architecture in which a language model acts as the central reasoning engine. It decides which actions to take, invokes external tools, and synthesizes results to fulfill user requests. Unlike simple question-answering, agentic behavior involves planning, acting, and adapting based on intermediate results.

2.2.1 Tool Calling

Tool calling is the ability of a language model to invoke external functions or APIs as part of its response. The model receives a set of available tool schemas, including names, descriptions, and parameter types. It must then select the appropriate tool for a request, generate valid arguments matching the schema, and incorporate the tool's response into its reasoning.

Tool calling is the primary capability evaluated in this thesis. It requires both language understanding (to map user intent to the correct tool) and structured generation (to produce syntactically and semantically valid function calls).

2.2.2 Structured Output Generation

Agentic systems typically require the model to produce outputs in a specific format, most commonly JSON. The output must satisfy three conditions: it must be syntactically valid, conform to a predefined schema, and contain semantically correct values for the given context.

Without explicit constraints, language models frequently produce malformed outputs: missing closing brackets, incorrect field names, wrong data types, or extra fields not present

in the schema. These failures are particularly common in small models and represent a key reliability bottleneck for agentic deployments.

2.2.3 The ReAct Pattern

ReAct (Reasoning and Acting) [34] is a prompting framework that interleaves reasoning traces with action steps. Rather than generating a final answer directly, the model follows a loop of Thought, Action, and Observation. The Thought step reasons about what to do next, the Action step calls a tool or performs a step, and the Observation step processes the result.

This pattern improves multi-step task completion by making the model's reasoning explicit and allowing it to adapt its plan based on intermediate results. ReAct is particularly relevant for agentic systems because it provides a structured way to combine reasoning and tool use within a single generation process.

Within the ReAct loop, the Action step is where the model must emit a structured tool call. For small models, this is also where generation often fails. The reasoning trace may be sound, but the emitted call may be malformed JSON or may name a function that does not exist.

This thesis isolates that step. Constrained decoding (Section 2.3) enforces structural validity at the Action step, so that a small model's reasoning is not discarded because of a formatting error. Chapter 4 evaluates whether this shifts the bottleneck from structural failure to residual semantic failure.

2.2.4 Cascade Architectures

A cascade architecture routes inference requests across two or more models of increasing capability and cost. Routine requests are handled by a smaller, faster model deployed locally; requests that exceed its reliable operating range are escalated to a larger model, typically accessed through an external API. The routing decision may be based on query complexity, output confidence, or post-hoc structural validation of the smaller model's response.

For tool-calling tasks, structural validity is a necessary but not sufficient condition for accepting a small model's response. A response that is syntactically valid JSON and names an existing function may still contain incorrect argument values. The residual semantic failure rate after structural constraints are enforced therefore determines the escalation boundary: queries whose failure mode is argument value errors cannot be filtered by structural checks alone.

This thesis is framed around extending the small model operating region within a cascade; the economic motivation and deployment context are described in Chapter 1.

2.3 Constrained Decoding

Constrained decoding modifies generation so that the model can only produce outputs that conform to a predefined structure. Instead of relying on the model to learn output formats implicitly, constrained decoding enforces them at inference time by masking invalid tokens at each generation step.

2.3.1 Formal Grammars for Generation

The key idea is to define the valid output space with a formal grammar, typically a context-free grammar (CFG) or a JSON schema. At each decoding step, the grammar determines which tokens are valid continuations given the tokens generated so far. Invalid

tokens receive probability zero, or negative infinity in log-space, so only grammatically valid sequences can be produced.

For tool calling, this means the model is guaranteed to produce valid JSON that conforms to the tool's argument schema. Field names, types, and structure are enforced by construction rather than learned behavior. Willard and Louf [31] formalized this approach using finite-state machines compiled from regular expressions and parsers for context-free grammars.

A context-free grammar specifies the valid output space as a set of production rules. A minimal grammar for a JSON object carrying a single integer field illustrates the idea:

```
object -> "{" '"a":' integer "}"
integer -> digit | digit integer
digit  -> "0" | "1" | ... | "9"
```

After the opening brace, the grammar admits only the literal `"a":`, so any token whose text does not extend that literal is masked. After the field marker, it admits a digit, and this continues until the closing brace. The per-step overhead is low because the partial output is not re-parsed at every step. Instead, the grammar is compiled once, before generation, into a finite-state machine. The model's tokenizer vocabulary is then aligned against that machine, so each state stores the token IDs that are valid continuations from it [31].

At generation time, the constraint is therefore a lookup of the current state's precomputed token mask. The cost does not grow with the size of the grammar; XGrammar [6] further caches these masks to reduce the remaining per-step cost. This indexing step is what libraries such as Outlines [7] perform, and it is why constrained decoding can enforce a non-trivial grammar with little added latency per token.

2.3.2 Guided Decoding Techniques

Several libraries implement constrained decoding for language models. Outlines [7] uses finite-state machines compiled from regular expressions or JSON schemas to guide generation. LMQL [2] combines constrained decoding with a query language for expressing complex output constraints declaratively. XGrammar [6] accelerates context-free grammar execution with a token-mask cache, reducing the per-step overhead of grammar-constrained generation.

Format constraints are not free in general. Tam et al. [27] report that strict format restrictions can degrade reasoning performance on tasks where free-form derivation helps. Chapter 4 examines this trade-off empirically for tool calling.

These techniques are particularly impactful for small models because they eliminate an entire class of errors (malformed outputs) without requiring any additional training. Constrained decoding is the base layer of this thesis, upon which the other techniques are stacked.

2.4 Prompt Engineering and Inference-Time Compute

2.4.1 Few-Shot Prompting

Few-shot prompting provides the model with examples of correct input-output pairs directly in the prompt [3]. For tool calling, this means including examples of user requests paired with the expected function calls. The model uses these examples as templates to guide its own generation.

Few-shot prompting is a zero-cost technique (no training required) that can improve task performance on format-learning and pattern-matching tasks. Its effectiveness depends on

whether the bottleneck is output format compliance or semantic convention matching; for the latter, in-context examples may not generalise across function domains.

2.4.2 Chain-of-Thought Reasoning

Chain-of-thought (CoT) prompting [30] encourages the model to generate intermediate reasoning steps before producing a final answer. Instead of directly outputting a tool call, the model first explains its reasoning: which tool is appropriate, what arguments are needed, and why.

CoT is a form of inference-time compute, trading additional generated tokens for improved accuracy. It is particularly effective for tasks that require multi-step reasoning, such as selecting among many similar tools or constructing complex function arguments. For small models, CoT can compensate for limited implicit reasoning capacity by making the reasoning process explicit.

2.5 Retrieval-Augmented Generation

Retrieval-augmented generation (RAG) [17] augments the model's input with relevant information retrieved from an external knowledge source at inference time. In the context of tool calling, RAG retrieves relevant tool schemas and documentation based on the user's request, rather than including all available tools in every prompt.

This approach offers two benefits for small models. First, it reduces the prompt length by only including relevant tools, which is important given the limited context windows of smaller models. Second, it reduces hallucination by grounding the model's generation in retrieved factual content rather than relying solely on parametric knowledge.

A typical RAG pipeline for tool calling involves encoding tool descriptions into a vector store, retrieving the top-k most relevant tools for a given query using semantic similarity, and injecting the retrieved schemas into the model's prompt before generation.

The retrieval step depends on a dense vector index. Each tool description is encoded into a fixed-dimensional embedding vector by a sentence encoder, and all embeddings are stored in an index that supports approximate nearest-neighbour search. At query time, the user's request is encoded by the same model, and the k tool embeddings with the highest cosine similarity to the query embedding are retrieved. FAISS (Facebook AI Similarity Search) is a widely used library for this purpose, providing efficient exact and approximate search over large embedding collections [14].

The primary failure mode of RAG for tool calling is candidate disambiguation. When multiple tools have semantically similar descriptions (for example, `calculate_area`, `calculate_triangle_area`, and `calc_area_triangle`), the retrieval step returns all of them as high-similarity candidates, and the language model must then select the correct one. This selection task requires understanding subtle distinctions in tool semantics that may not be apparent from descriptions alone. The quality of retrieval is measured by recall@k: the fraction of queries for which the correct tool appears among the k retrieved candidates. High recall@k is necessary but not sufficient for accurate tool calling; the model must still identify the correct tool from the retrieved set.

2.6 Parameter-Efficient Fine-Tuning

Fine-tuning adapts a pre-trained model to a specific task by continuing training on task-specific data. Full fine-tuning updates all model parameters, which is computationally expensive and requires significant memory. Parameter-efficient fine-tuning (PEFT) methods address this by updating only a small subset of parameters while keeping the rest frozen.

2.6.1 LoRA

Low-Rank Adaptation (LoRA) [12] is the most widely used PEFT method. Instead of updating the full weight matrices during fine-tuning, LoRA injects trainable low-rank decomposition matrices alongside the frozen pre-trained weights. For a weight matrix $W \in \mathbb{R}^{d \times k}$, LoRA learns two smaller matrices $A \in \mathbb{R}^{d \times r}$ and $B \in \mathbb{R}^{r \times k}$, where $r \ll \min(d, k)$, such that the adapted weight becomes $W + AB$.

This reduces the number of trainable parameters by orders of magnitude while achieving performance comparable to full fine-tuning on many tasks. For tool calling, LoRA can be used to fine-tune small models on datasets of function-calling examples, teaching them to select tools and generate arguments more accurately.

2.6.2 PEFT Variants

Beyond LoRA, several other PEFT methods exist. QLoRA [5] combines LoRA with quantization, applying low-rank adaptation on top of a 4-bit quantized base model to further reduce memory requirements. Adapter layers insert small trainable modules between frozen Transformer layers. Prefix tuning prepends trainable continuous vectors to the key and value sequences in each attention layer.

This thesis primarily uses LoRA due to its simplicity, strong empirical performance, and wide support across frameworks and model architectures.

2.7 Model Quantization

Quantization reduces the numerical precision of model weights from the 16-bit or 32-bit floating-point formats used during training to lower-bit integer representations at inference time. A weight stored as a 4-bit integer requires one quarter of the memory of a 16-bit floating-point value, enabling models that would otherwise require multi-GPU setups to run on a single consumer GPU. Secondary benefits include faster weight loading from storage and reduced memory bandwidth during inference.

Post-training quantization (PTQ) applies the precision reduction after training is complete, without additional gradient computation. This contrasts with quantization-aware training (QAT), which simulates quantization during training and achieves higher accuracy at the cost of retraining. PTQ is preferred for large language models because the cost of retraining at scale is prohibitive; the challenge is to maintain accuracy despite the information loss from reduced precision.

2.7.1 Activation-Aware Weight Quantization

Naive PTQ at 4-bit precision produces significant accuracy degradation because not all weights tolerate uniform quantization equally. Weights connected to channels with large activation magnitudes have a disproportionate effect on output quality; quantizing these salient weights to 4 bits introduces large relative errors where the model is most sensitive.

Activation-aware Weight Quantization (AWQ) [18] identifies salient channels by examining activation statistics on a small calibration dataset and applies a per-channel scaling transformation that reduces the effective quantization error for high-magnitude channels. The weights are then quantized uniformly to INT4 after scaling. The transformation is absorbed into the preceding layer weights, so inference requires no additional computation beyond standard INT4 matrix multiplication.

For the Qwen 2.5 7B-Instruct model used in this thesis, AWQ INT4 quantization reduces model memory from 14.25 GiB to 5.20 GiB, a 63.5% reduction, and reduces model loading time from 212 s to 35 s. The accuracy cost on BFCL v4 `simple_python` is -0.5 percentage

points relative to the bfloat16 baseline, within run-to-run variance. Pre-quantized AWQ INT4 weights are published on the Hugging Face Hub for all Qwen 2.5 Instruct sizes [24], avoiding the need for manual quantization.

2.8 Related Work

2.8.1 Tool Calling and Benchmarks

Function-calling benchmarks span a spectrum that trades catalogue size against task depth. The Berkeley Function Calling Leaderboard (BFCL) [32] is the standardized single-turn benchmark on which the configurations in this thesis are evaluated; its categories, metrics, and the specific test splits used are described with the experimental setup in Section 3.2. It is the reference point against which the other benchmarks below are positioned.

ToolLLM [23] extends function-calling evaluation to over 16,000 real-world APIs from the RapidAPI catalogue, covering domains including finance, weather, social media, and e-commerce. At this scale, the model must select among thousands of APIs with varying documentation quality and compose multi-step tool chains. The primary metric is execution accuracy, requiring correct runtime results rather than structural matching. The scope of ToolLLM reflects a harder generalization challenge than BFCL, as the API catalog far exceeds what can be memorized from pre-training data.

At the multi-step end of the spectrum, the τ-bench benchmark [33] evaluates end-to-end agentic task completion in simulated retail and airline customer service domains, scoring only whether the final database state matches a ground truth resolution after a sequence of interdependent tool calls. This thesis uses τ-bench to probe multi-step reliability beyond single-call accuracy; its task structure, scoring, and the user-simulator configuration are detailed in Section 3.2.

2.8.2 Fine-Tuned Small Models for Tool Use

Gorilla [22] demonstrated that a LLaMA model fine-tuned on API documentation can outperform GPT-4 on API calling tasks. The key finding is that training data relevance can compensate for parameter count disadvantage when the task requires structured generation against specific schemas. Gorilla uses a retrieval-augmented training procedure in which API documentation is retrieved during training, teaching the model to use retrieved context rather than rely on memorized API signatures.

The xLAM model family [35] extends the fine-tuning approach with a large-scale training pipeline that draws from diverse function-calling datasets. The `xlam-function-calling-60k` dataset used in this thesis for LoRA fine-tuning was produced by the APIGen pipeline [19], which generates and automatically verifies function-calling examples across a wide range of API types and domains. The diversity is intended to improve generalization beyond the specific schemas seen during training. Models trained on this dataset are competitive with frontier systems on standard tool-use benchmarks.

The format of training outputs has a direct effect on how fine-tuning interacts with constrained decoding at inference time. This interaction is observed in the present experiments: two LoRA adapters trained on the same dataset with identical hyperparameters but different output formats converge to similar final training loss (0.203 and 0.229 at step 6,750 respectively) yet differ by 7.0 percentage points in BFCL accuracy when paired with the constrained decoding pipeline. The accuracy difference is attributable entirely to whether the training output format matches the inference-time generation target, not to any difference in training quality. This result extends Gorilla's data-relevance insight from API source selection to output format alignment.

A recent survey of small language models for agentic systems [25] identifies Qwen 2.5 7B as one of the strongest open-weight SLMs for tool use, attributing its performance to instruction tuning on function-calling examples and the scale of its pre-training corpus. The survey notes that most existing evaluations test individual optimization techniques in isolation, and that systematic combinations of constrained decoding, quantization, and fine-tuning remain underexplored. This gap directly motivates the cumulative evaluation design adopted in this thesis.

2.8.3 Constrained Generation

Willard and Louf [31] formalized constrained generation using finite-state machines (FSMs) compiled from regular expressions and JSON schemas. At each decoding step, the current grammar state determines the set of valid next tokens; tokens outside this set receive a logit of negative infinity before sampling, ensuring that the generated sequence conforms to the grammar by construction. The key contribution is an efficient offline compilation algorithm that converts JSON schemas into FSMs before inference begins, so that the per-step cost at generation time consists only of a state transition lookup.

The Outlines library [7] implements the FSM approach and is the constrained generation backend used in this thesis via vLLM [16]. Two generation modes are relevant to the two-stage tool-calling pipeline. The `guided_choice` mode restricts output to one string from a predefined list and is used in Stage 1 to guarantee that the predicted function name exists in the available schema. The `guided_json` mode restricts output to a JSON object conforming to a provided schema and is used in Stage 2 to guarantee structural validity of the argument object. Both modes compile their constraint to a token mask at request initialization time, amortizing the compilation cost across all generation steps for a given request.

The per-step overhead of FSM-based constrained generation is low when schemas are compiled ahead of time. Overhead scales with schema complexity: schemas with deeply nested structures and many optional fields require larger FSM state spaces and more expensive state transitions. For the function schemas in BFCL `simple_python`, complexity is moderate and the overhead falls within practical inference budgets. Small models benefit disproportionately from constrained generation because their baseline format error rates are higher: without constraints, Qwen 2.5 7B produces unparseable output on 98.5% of BFCL test cases, a failure rate that constrained decoding eliminates entirely.

LMQL [2] offers an alternative approach through a declarative query language in which output constraints are expressed as Python predicates evaluated during generation. The approach is more flexible than FSM-based methods for constraints that are difficult to specify as formal grammars. The per-step cost is higher, however, because predicates are evaluated at each generation step rather than looked up from a precompiled state table. This thesis uses FSM-based constrained decoding via vLLM because it is better integrated with batch inference and provides formal guarantees from a compiled grammar state.

2.8.4 Cascade Architectures

A cascade architecture routes inference requests across models of different capability and cost. Routine requests are handled by a smaller, faster model; requests that exceed its reliable operating range are escalated to a larger model, typically accessed through an external API. Routing decisions may be based on output confidence, output validation, or a query complexity estimate computed before inference.

For tool-calling tasks, structural validation is a natural routing gate. If the small model output fails to parse as valid JSON or names a function not present in the available schema,

the response can be rejected and escalated without execution. Constrained decoding removes this failure mode by construction, since every output is structurally valid by guarantee. After constrained decoding is applied, the residual failure rate is attributable entirely to incorrect argument values, which structural checks cannot detect. This shifts the routing problem from format detection to semantic verification, a harder problem that requires either ground-truth access or execution feedback.

The residual semantic failure rate determines the fraction of traffic that must be escalated to a frontier model and the associated variable inference cost. CD achieves 72.75% accuracy, implying a 27.25% escalation rate under structural routing. Config CD+FT-aligned reduces this to 23.25%, a reduction of approximately 4.0 percentage points. The relationship between optimization technique selection and cascade routing cost is a practical design consideration that has received limited attention in prior work; this thesis provides direct measurements for five technique combinations.

2.8.5 Small Model Optimization

Recent model families have narrowed the capability gap with frontier models through improvements applied at training time. Phi-4 [1] achieves competitive performance with models several times its size on reasoning and mathematics benchmarks, demonstrating that training data curation can compensate for reduced parameter count. Llama 3 [9] introduces grouped-query attention and an expanded vocabulary to improve inference efficiency and multilingual coverage. Qwen 2.5 [24] achieves strong tool-use performance through instruction tuning on function-calling examples and a context window of 128,000 tokens.

Knowledge distillation [10] provides a complementary path to capable small models. A small student model is trained to match the output distribution of a large teacher rather than ground-truth labels, transferring reasoning capabilities that the student would not acquire from labeled data alone. Unlike the post-training techniques evaluated in this thesis, distillation affects the model's parametric knowledge and requires access to the teacher model's outputs during training.

Quantization reduces model memory at the cost of a small accuracy penalty. For the Qwen 2.5 7B model used in this thesis, AWQ INT4 quantization [18] reduces memory from 14.25 GiB to 5.20 GiB and reduces accuracy by 0.5 percentage points on BFCL `simple_python`, a cost within run-to-run variance. This result is consistent with reported accuracy retention for AWQ across other model families at similar parameter scales.

The pre-training and instruction-tuning improvements embodied in Phi-4, Qwen 2.5, and Llama 3 are largely orthogonal to the inference-time and post-training techniques evaluated here. Constrained decoding, quantization, and LoRA fine-tuning can be applied on top of any pre-trained checkpoint. The question addressed in this thesis is how much additional accuracy is recovered by combining these techniques on top of an already well-trained small model, and which combinations offer the best accuracy-cost tradeoff for deployment in a cascade architecture.

3 Methodology

This chapter describes how the optimization techniques are evaluated. The experiments follow a cumulative ladder: a baseline configuration is established first, and one technique is added at each subsequent step to isolate its marginal contribution. The Qwen 2.5 model family is evaluated across four sizes from 0.5B to 7B. The techniques covered are constrained decoding, schema enrichment, quantization, few-shot prompting, chain-of-thought prompting, retrieval-augmented generation, and LoRA fine-tuning. All configurations are evaluated on the Berkeley Function Calling Leaderboard.

3.1 Experimental Design

The experimental design is based on controlled comparisons and runs in two complementary directions. In the first, a baseline configuration is established and each technique is added one at a time, which measures the marginal gain of each technique on top of the existing stack. In the second, the inference-time techniques are evaluated without constrained decoding, which measures what each technique achieves on its own and quantifies the contribution of constrained decoding itself.

3.1.1 Cumulative Evaluation Ladder

Twelve configurations are evaluated in total. Eleven form the cumulative ladder in Figure 3.1; the twelfth, CD+schema, branches off separately and is reported in Section 4.2.3.

The ladder is organised around a main chain (Config B → CD → CD+Q → CD+Q+FT-aligned) that starts from an unconstrained full-precision baseline and adds one technique at each step: constrained decoding, then AWQ INT4 quantization, then a format-aligned LoRA adapter. The remaining configurations branch from this chain to isolate a single design choice rather than to extend the stack. Few-shot prompting (PE) branches from CD at full precision, so its effect is separated from quantization; chain-of-thought (ITC) and retrieval (RAG) branch from CD+Q; and two LoRA adapters, misaligned (CD+FT) and format-aligned (CD+FT-aligned), branch from CD. Two further variants drop constrained decoding entirely (FT-only and FT-aligned-ng, where ng denotes no guided decoding) to measure fine-tuning on its own.

The second arm of the design re-evaluates the three inference-time techniques without guided decoding (Config PE-ng, ITC-ng, and RAG-ng) to quantify the contribution of constrained decoding itself. These isolation variants are reported in Section 4.2.7; they are not rungs on the ladder and are omitted from Figure 3.1.

3.1.2 Hardware and Infrastructure

All experiments are conducted on the DTU HPC cluster (gbar.dtu.dk), running AlmaLinux 9.7 under the LSF job scheduler. Each job is allocated one NVIDIA L40S 48 GB GPU in exclusive process mode; inference jobs use four CPU cores and 32 GB of RAM, while LoRA training jobs use eight cores and 64 GB. Inference jobs have a two-hour wall-time limit and training jobs an eight-hour limit.

Model inference is served via vLLM 0.8.5, which exposes a REST API on localhost that the evaluation client connects to, with GPU memory utilisation limited to 90 %. Full-precision models are loaded in bfloat16 (14 GB VRAM for the 7B model); quantized models are loaded in float16 under the AWQ Marlin backend (4–5 GB VRAM). The software

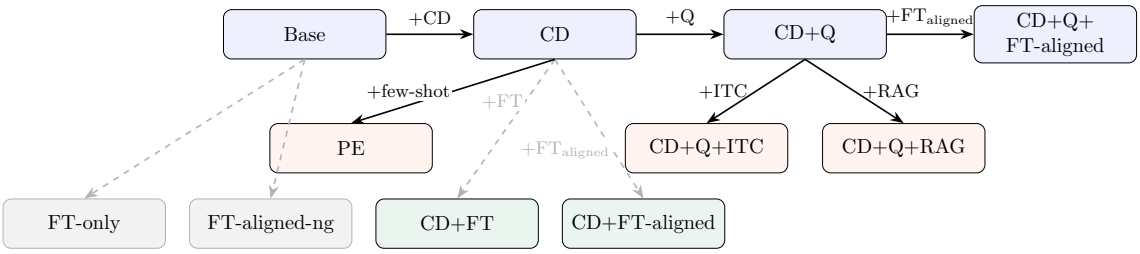

Figure 3.1: Incremental evaluation ladder showing eleven of the twelve configurations. The main chain (top row, solid arrows) adds one technique at a time. Orange nodes are inference-time branches: PE branches from CD at full precision; ITC and RAG branch from CD+Q. Green nodes are fine-tuning branches from CD. Gray nodes are unguided standalone configurations (no constrained decoding). Not shown: the off-ladder CD+schema variant (Section 4.2.3) and the no-guided isolation variants PE-ng, ITC-ng, and RAG-ng (Section 4.2.7), which are evaluated separately.

environment uses uv 0.5.13, Python 3.12.11, and CUDA 12.6.3. Model weights are cached from the Hugging Face Hub on cluster storage.

The evaluation pipeline, prompt templates, guided decoding schemas, HPC job scripts, and training scripts are available in the accompanying repository.[1]

3.2 Evaluation Framework

Two benchmarks are used to evaluate the configurations. The Berkeley Function Calling Leaderboard covers single-turn tool-calling accuracy across categories of increasing complexity. τ-bench covers multi-step agentic task completion, where the model must reason, call tools, observe results, and iterate until a task is done.

3.2.1 Berkeley Function Calling Leaderboard

All configurations are evaluated on the Berkeley Function Calling Leaderboard (BFCL) v4 [32]. Four categories are used: `simple_python` (400 test cases, one candidate function per test), `multiple` (the model must select the correct function from several candidates), `parallel` (the model must produce multiple simultaneous calls to the same function), and `parallel_multiple` (simultaneous calls to different candidate functions). Each test case consists of a user prompt and one or more function definitions; the model is required to produce the correct function call with the correct arguments. Frontier model scores are taken from the published BFCL leaderboard rather than re-evaluated, as the official scores are the canonical reference for the benchmark.

The primary metric is AST accuracy: the model output is parsed and compared structurally against the ground truth, checking function name, argument names, argument values, and argument types. BFCL also defines an execution accuracy metric, which runs the generated call and compares its runtime result against a reference output; AST accuracy is used throughout this thesis because it is deterministic, reproducible, and independent of the runtime environment, consistent with the leaderboard convention for comparing configurations systematically. Evaluation is performed using the BFCL `ast_checker`, which applies lenient type coercion between compatible types (e.g., integer and float). All inference runs use temperature 0 to ensure deterministic outputs across configurations; at temperature 0 vLLM applies greedy decoding, producing identical outputs on repeated runs within a fixed software environment. The CD+Q result (289/400) was confirmed

[1] https://github.com/pabeckha/slm-agents

across six consecutive re-runs that produced identical predictions. Re-runs spread over the project period, however, varied by one to two cases out of 400 (CD scored between 289 and 291, and CD+Q between 288 and 289, across runs months apart), so any single accuracy figure carries a run-to-run uncertainty of approximately ±0.5 pp. Headline figures quote the originally recorded runs; the paired statistical tests in Section 4.4 are computed on the stored per-case prediction files. Function selection accuracy and argument extraction accuracy are reported separately to identify where failures occur along the prediction pipeline.

To measure per-call inference latency, a sequential latency run (parallel dispatch disabled) is conducted on the first 100 `simple_python` test cases for three configurations: B, CD, and CD+Q. Sequential dispatch ensures that each measurement is a clean per-call latency without parallelism overlap. The same vLLM server and hardware are used as for the accuracy runs. Mean, median, and 95th-percentile latency are reported.

3.2.2 τ-bench

The τ-bench retail benchmark [33] evaluates end-to-end multi-step task completion. Each task is a simulated customer service interaction in which the model must reason across 5–15 tool calls, observe intermediate results, and reach a final database state that matches a ground truth resolution. CD and CD+FT-aligned are evaluated on the retail domain test split (115 tasks); the remaining configurations are not evaluated on τ-bench because the primary bottleneck identified is multi-turn planning rather than single-call argument accuracy.

Three full evaluation runs are completed to account for variance introduced by the user simulator. The user simulator is Qwen 2.5 7B-Instruct served on the same vLLM instance as the agent model, running at temperature 0.3 to produce naturalistic user responses. The agent model runs at temperature 0.0 to ensure deterministic tool call generation. All runs are executed on a single NVIDIA L40S GPU (48 GiB) on the DTU HPC cluster. This simulator choice departs from the reference τ-bench setup, which drives the user simulator with a frontier model; a 7B simulator may misstate or omit task details, which deflates pass rates independently of agent capability. The implications are discussed in the threats to validity (Section 5.3.1).

Scoring is binary: a task receives reward 1.0 only if the final database state exactly matches the ground truth at the end of the interaction, and reward 0.0 otherwise. Partial task completion yields no reward. The pass rate is reported as the mean reward across all tasks and all runs.

3.3 Models

3.3.1 Qwen 2.5

The Qwen 2.5 model family [24] is used across all experiments. Four sizes are evaluated: 0.5B, 1.5B, 3B, and 7B parameters, all in the Instruct variant. The 7B model is the primary subject of analysis; the smaller sizes are included to study how model scale interacts with each optimization technique.

Qwen 2.5 is selected for three reasons. First, the family covers a continuous size range under a single architecture and training regime, which allows scale effects to be isolated without confounding differences in model design. Second, native AWQ INT4 quantized variants are available for all sizes on the Hugging Face Hub, avoiding the need for manual quantization. Third, recent surveys of SLMs for agentic systems identify Qwen 2.5 7B as one of the strongest open-weight models for tool use and function calling at this parameter scale [25]. All models are served under an Apache 2.0 licence.

All Qwen 2.5 models share a decoder-only Transformer architecture with grouped-query attention (GQA), SwiGLU feed-forward activations, and rotary position embeddings (RoPE). The 7B variant uses 28 layers, a hidden dimension of 3584, and 28 query heads grouped into 4 key-value head groups. The supported context length is 128,000 tokens, though all experiments in this thesis operate well within this limit. GQA reduces the KV cache memory footprint relative to standard multi-head attention, which is one reason the 7B model fits comfortably on a single consumer GPU in bfloat16 precision.

The Instruct variant is pre-trained on a multilingual corpus of approximately 18 trillion tokens. Supervised fine-tuning on instruction-following and tool-use examples, followed by alignment via reinforcement learning from human feedback, produces a model that follows instructions reliably. It responds to structured prompts and tool schemas without additional task-specific training, serving as a practical starting point for the optimization ladder evaluated in this thesis.

3.3.2 Contrast Checkpoints

The cross-family checks in Chapter 4 use four additional instruct models from three other families: Llama 3.2 1B-Instruct and Llama 3.2 3B-Instruct [20], Phi-4-mini-Instruct [1], and Gemma 3 1B-Instruct [8]. These are contrast checkpoints rather than part of the primary size ladder. Their purpose is to test whether the findings established on Qwen 2.5 depend on one model's training or hold under different pre-training corpora and instruction-tuning recipes.

The four models span roughly 1B to 4B parameters: Gemma 3 1B and Llama 3.2 1B sit near 1B, Llama 3.2 3B at 3.2B, and Phi-4-mini at 3.8B, overlapping the lower half of the Qwen range. Their training regimes differ in ways that matter for this check. Phi-4 emphasises curated and synthetic training data, the small Llama 3.2 models are distilled from larger Llama checkpoints, and Gemma 3 is likewise trained with distillation. A result that survives all four is unlikely to be an artefact of any single training pipeline.

The contrast checkpoints enter the evaluation in three places. The constrained-decoding generalization check (Section 4.8.1) evaluates Config CD on BFCL v4 `simple_python` for each checkpoint, to test whether the structural benefit established on Qwen holds under different pre-training corpora. The format-aligned fine-tuning analysis (Section 4.4.1) trains a format-aligned adapter on each model and evaluates it without guided decoding, to check whether the standalone degradation seen on Qwen also appears across families. The GitHub MCP evaluation (Section 4.7) compares baseline and constrained tool selection across all five families on the 21-tool catalogue. The quantized, fine-tuned, and τ-bench configurations stay on Qwen 2.5; extending them across the contrast families is left to future work. Llama 3.2 is served under the Llama 3.2 Community License, Phi-4-mini under the MIT License, and Gemma 3 under the Gemma Terms of Use.

3.4 Baseline Definition

Base is the baseline configuration: the model is prompted without constrained decoding, at full precision (bfloat16), with no quantization, no fine-tuning, and no few-shot examples. The pipeline is two-stage (Figure 3.2). In the first stage, the available function signatures and descriptions are presented to the model, and the prompt ends with `Function name:` ; the model completes freely. In the second stage, the selected function signature and the user query are presented, and the prompt ends with `JSON:` ; the model generates argument values as free-form text. A JSON extraction heuristic is applied to the output, capturing the substring from the first opening brace to the last closing brace. Temperature is set to 0 across all configurations to ensure deterministic outputs, removing randomness as a

confound when comparing techniques.

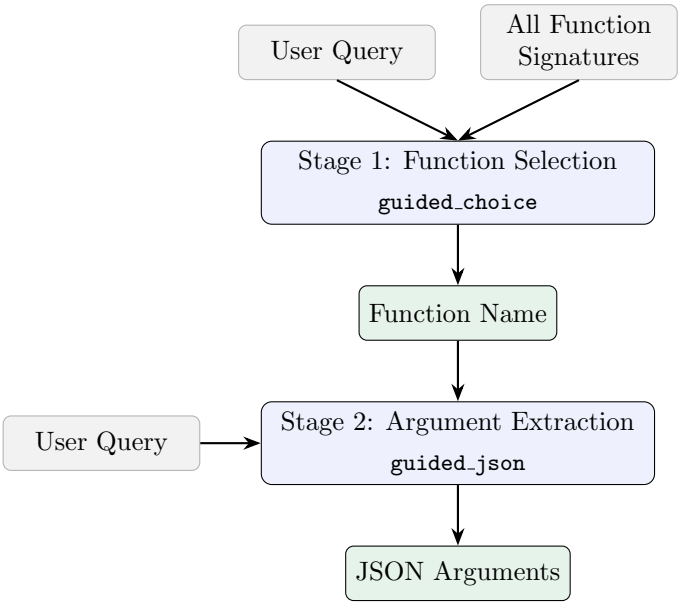

Figure 3.2: Two-stage inference pipeline. Stage 1 selects a function name from all available candidates; Stage 2 extracts argument values for the selected function. In CD and above, `guided_choice` and `guided_json` enforce structural validity at each stage.

Base establishes the lower bound for the evaluation ladder: the model is presented with the generic, model-agnostic pipeline and no constrained decoding, so any improvement at a later rung is attributable to the technique added there. Its accuracy and failure breakdown are reported in Chapter 4.

The argument-extraction stage parses the completion as JSON under two criteria. The headline baseline applies a strict whole-completion parse: the entire model output must load as a single JSON object, which is the floor a naive model-agnostic integration achieves (1.50%). The technique-isolation arm (Section 4.2.7) instead reads the first complete JSON object and ignores trailing text, so the prompt-level techniques are compared fairly; under this lenient criterion the same Base run scores 62.00%. An earlier no-guided parser sliced from the first to the last brace and so failed on a valid answer followed by further text; it was replaced by the first-object parser used for all corrected no-guided figures.

Base is deliberately model-agnostic: it does not use the tool-calling prompt template that Qwen 2.5 provides. The 1.5% figure measures what a generic integration achieves without model-specific prompting, which is the relevant floor for the technique ladder evaluated here: every subsequent configuration changes exactly one component relative to this fixed pipeline.

A control configuration, B-template, bounds this design choice from above. It prompts the same model through its native tool-calling chat template, passing the full unmodified BFCL function schemas (including parameter descriptions, default values, and enumerations), and parses the model's native `<tool_call>` output with no guided decoding. Two properties separate it from Base: the prompt format matches the model's instruction tuning, and the full schema detail is visible to the model, whereas the model-agnostic pipeline carries only parameter names and types. In exchange, B-template provides no structural validity guarantee and applies only to models that ship a tool-calling template. Its accuracy is reported in Chapter 4, and the implications of this control for interpreting the residual

failure rate of the constrained configurations are discussed in Chapter 5.

Figure 3.3 makes the two designs concrete for the `add` query from Chapter 1. The model-agnostic pipeline (a) anchors generation with the plain-text `JSON:` cue and reduces each parameter to a `name (type)` pair, so the model receives neither the format it was instruction-tuned on nor the parameter descriptions, defaults, and enumerations present in the source schema. The native template (b) renders the same request the way the model saw tool use during training: the full JSON schema is placed inside a `<tools>` block in the system message, and the model is asked to wrap its call in `<tool_call>` tags, which the backend then parses. The contrast isolates the two confounds folded into the Base score, prompt format and schema completeness, from the model's underlying capability.

3.5 Constrained Decoding

CD adds constrained decoding to Base; all other settings remain unchanged. Constrained decoding is implemented via vLLM's guided decoding feature [16, 31], which applies logit masking at each generation step to restrict the output to a valid token sequence.

Two mechanisms are used in the two-stage pipeline. In the function selection stage, `guided_choice` restricts the output to one of the valid function names, guaranteeing that the selected name exists in the available function set. In the argument extraction stage, `guided_json` restricts the output to a JSON object that conforms to a schema derived from the function definition, with parameter names, types, and required fields enforced, and `additionalProperties` set to false.

For the `parallel` and `parallel_multiple` categories, which require several simultaneous calls, the single-name selection stage is replaced. The initial design generated a JSON object of the form `{"calls": [...]}` under `guided_json`, where the array items were restricted to the set of candidate function names and the maximum array length equalled the number of distinct candidates; each selected name was then passed through the standard argument extraction stage. This design emits at most one argument object per distinct function name: because argument extraction is conditioned only on the user query and the selected function, and decoding is deterministic at temperature 0, two calls to the same function would receive identical argument objects. Every `parallel` case provides exactly one distinct candidate function, so this design produces a single call by construction.

To support repeated calls to the same function, the schema is extended to a joint multi-call form: rather than selecting a list of names and then extracting arguments per name, the model generates a JSON array of complete call objects in a single guided-decoding pass, where each object carries its own function name and argument bindings and the same function may recur with distinct arguments. Both schemas are evaluated. The single-extraction design defines the boundary of the per-function pipeline, and the joint multi-call schema is the remedy for it; the consequences of each for the `parallel` category are analysed in Section 4.9 and Chapter 5.

Figure 3.4 contrasts the two schemas. The single-extraction design selects a list of distinct function names and then extracts arguments for each name independently, so it emits at most one argument object per distinct name. The joint multi-call schema instead produces a single JSON array of complete `{name, arguments}` objects, in which the same function may appear more than once with different arguments.

The resulting accuracy for CD, together with the function-selection rate and the breakdown of residual argument-value failures, is reported in Chapter 4.

3.6 Quantization

CD+Q adds AWQ INT4 quantization [18] to CD. The pre-quantized model `Qwen/Qwen2.5-7B-Instruct-AWQ` is used, loaded via vLLM with the `awq_marlin` backend, which applies optimized Marlin kernels for efficient INT4 inference on GPU. All other settings remain unchanged.

AWQ is selected because activation-aware weight quantization preserves accuracy better than uniform quantization by protecting weights that correspond to large activation values. Pre-quantized variants for all Qwen 2.5 sizes are available on the Hugging Face Hub, avoiding the need for a local quantization step.

CD+Q achieves 72.25% AST accuracy (289/400), a reduction of 0.5 pp relative to CD, within run-to-run variance. AWQ INT4 quantization reduces the 7B model's GPU memory footprint from 14.25 GiB to 5.20 GiB (63.5% reduction) and startup time from 212 s to 35 s. No new failure modes are introduced: the failure distribution remains semantic, with the same categories of argument value errors as CD.

3.7 Prompt Engineering and Few-Shot

PE extends CD by prepending three few-shot examples to the argument extraction prompt. The function selection prompt is left unchanged; examples are added only to the argument extraction stage, where all CD failures occur.

The three examples are constructed to target the most frequent failure categories observed in CD by inspecting the full set of 109 incorrect predictions on the test split. The examples are not drawn from the test split itself; they are hand-authored to illustrate the identified failure classes. This design choice is a limitation: the failure categories are identified from the same test split used to evaluate PE, so the examples are informed by test-set error patterns rather than a held-out development set. The first example demonstrates numeric precision: the expected value is `9.81`, not the rounded approximation `9.8`. The second demonstrates string format with a location qualifier: the expected value is `"San Francisco, CA"`, not `"San Francisco"`. The third demonstrates Python expression syntax: the expected value is `x**2`, not the mathematical notation `x^2`. Three examples are selected as the minimum number needed to cover distinct failure classes; additional examples would increase context length without introducing new failure categories.

All other settings remain unchanged from CD: the full-precision Qwen 2.5 7B-Instruct model is used, guided decoding is applied to both pipeline stages, and temperature is set to 0. PE is evaluated as a parallel track to CD+Q rather than as an extension of it, so the full-precision model is retained to isolate the effect of the few-shot examples from any interaction with quantization.

3.8 Inference-Time Compute

CD+Q+ITC extends CD+Q with a chain-of-thought reasoning step [30, 26]. The two-stage pipeline is unchanged at its endpoints: function selection still uses `guided_choice` and argument extraction still uses `guided_json`. A free-form reasoning stage is inserted between them. After the function name is selected, the model generates a reasoning trace of up to 256 tokens, prompted to set out which arguments the call requires and what values they should take, before the final guided extraction.

The reasoning stage is deliberately unconstrained. The structural guarantee of constrained decoding applies only to the final extraction, where it is needed to produce a valid call; the intermediate trace is left free so the model can deliberate about argument values without

the token mask restricting it. The 256-token budget bounds the additional inference cost while leaving room for the model to reason over the argument set of a single call. The reasoning text is discarded after the extraction step and does not enter the parsed output.

CD+Q+ITC branches from CD+Q rather than extending the stack, so the AWQ INT4 model is retained and the only change relative to CD+Q is the inserted reasoning stage. All other settings are inherited from CD+Q: guided decoding on both endpoints and temperature 0. Isolating the effect of inference-time compute from quantization at the 7B scale, where the quantization penalty is 0.5 pp, is not required because the reasoning stage is evaluated as a single addition to the CD+Q operating baseline.

3.9 Retrieval-Augmented Generation

CD+Q+RAG extends CD+Q by replacing the per-test-case oracle function provision with a retrieval step [17]. In all prior configurations, each test case is presented with the exact function definition required to answer it. In CD+Q+RAG, the model receives instead the top-k functions retrieved from an index of the full function catalogue.

All 370 unique function signatures from the `simple_python` category are embedded using `sentence-transformers/all-MiniLM-L6-v2` and indexed with FAISS before evaluation begins. The `all-MiniLM-L6-v2` model is selected because it is designed for semantic similarity of short texts, requires no GPU, and is approximately 80 MB, making it compatible with the vLLM server that holds the GPU in exclusive process mode. The embedder is run on CPU to avoid a device ownership conflict with vLLM.

At inference time, the user query is embedded and the top-5 most similar functions are retrieved as candidates ($k = 5$). The value $k = 5$ is selected to achieve near-complete recall while keeping the candidate set small enough for the model to disambiguate: a smaller k risks excluding the correct function, while a larger k increases the number of semantically similar alternatives the model must distinguish. The model selects among the retrieved candidates using `guided_choice` restricted to the five retrieved function names, then extracts arguments using `guided_json` as in CD+Q. All other settings are inherited from CD+Q.

3.10 LoRA Fine-Tuning

Two fine-tuning configurations are evaluated: FT-only, which applies the trained model without constrained decoding, and CD+FT, which combines the trained model with the constrained decoding pipeline from CD. The split between the two variants allows the independent contributions of fine-tuning and constrained decoding to be measured separately.

The base model is `Qwen/Qwen2.5-7B-Instruct` in bfloat16. A LoRA adapter is trained for 2 epochs with rank 16 on the Salesforce `xlam-function-calling-60k` dataset [35], using 54,000 training examples and 6,000 validation examples, with the HuggingFace PEFT library and TRL SFTTrainer [12].

Rank 16 is selected as a mid-range value for instruction-following adaptation: it provides sufficient capacity for the model to learn format and value conventions from the training data without overfitting. Two epochs are selected because the dataset is large enough that a single pass provides substantial exposure to the training distribution; a third epoch produced no further improvement in validation loss in preliminary runs.

After training, the adapter weights are merged into the base model by linear interpolation, producing a single merged checkpoint in bfloat16. The merged model is used for all

evaluation runs, removing adapter inference overhead and ensuring that the evaluation infrastructure is identical to that used for Base and CD.

3.11 Schema-Enriched Constrained Decoding

Config CD+schema extends CD by enriching the argument extraction prompt with the full parameter schema detail available in the BFCL function definitions. In all preceding configurations, the Stage 2 `guided_json` prompt carries only parameter names and types. Config CD+schema additionally injects, for each parameter, its natural-language description, default value (if any), and enumeration constraints (if any), drawn directly from the BFCL function schema.

The structural constraint is unchanged. `guided_json` continues to enforce JSON object validity and type compliance at each generation step via the same finite-state machine as CD. The schema enrichment acts on the prompt only: it provides the model with richer semantic context for value selection without altering the token-level mask. The two mechanisms are composable and independent.

Figure 3.5 shows the difference on a single test case (`simple_python_34`). The template prompt lists each parameter only as `name (type)`; the enriched prompt adds a `Parameters:` block carrying the natural-language description, the allowed values, and the default for each parameter. Here the user asks to "prefer exploring nature" while the `exploration_type` parameter is an enumeration whose members (`nature`, `urban`, `history`, `culture`) appear only in the enriched prompt: under the template prompt the model must guess the exact enum spelling, whereas the enriched prompt supplies it.

Config CD+schema is evaluated on the full 400-case `simple_python` test set across all four Qwen 2.5 sizes for the size-scaling analysis. The primary paired significance test is reported at 7B, where a paired McNemar test on stored per-case predictions compares CD+schema against the stored CD run (290/400) used as the paired baseline. The size sweep shows whether the schema-enrichment effect persists across capacity, while the paired 7B comparison isolates the contribution of schema information to the residual failure rate after constrained decoding.

(a) Base — generic, model-agnostic pipeline (Stage 2 prompt)

```
Function: add(a: number, b: number) -> number
Description: Add two numbers
User request: What is the sum of 265 and 345?
Extract the argument values as a JSON object with keys: a (number), b (number).
JSON:
```

Model output (free generation; strict whole-completion parse fails):

```
{"a": 265, "b": 345} The function ‘add‘ takes two numbers as arguments
and returns their sum [...] So, the sum of 265 and 345 is 610.
```

(b) B-template — native tool-calling chat template (rendered prompt)

```
<|im_start|>system
You are Qwen, created by Alibaba Cloud. You are a helpful assistant.

# Tools

You may call one or more functions to assist with the user query.

You are provided with function signatures within <tools></tools> XML tags:
<tools>
{"type": "function", "function": {"name": "add", "description": "Add two
numbers", "parameters": {"type": "object", "properties": {"a": {"type":
"number"}, "b": {"type": "number"}}, "required": ["a", "b"]}}}
</tools>

For each function call, return a json object with function name and arguments
within <tool_call></tool_call> XML tags:
<tool_call>
{"name": <function-name>, "arguments": <args-json-object>}
</tool_call><|im_end|>
<|im_start|>user
What is the sum of 265 and 345?<|im_end|>
<|im_start|>assistant
```

Model output (parsed from the `<tool_call>` block):

```
<tool_call>
{"name": "add", "arguments": {"a": 265, "b": 345}}
</tool_call><|im_end|>
```

Figure 3.3: The same `add` request under the two baseline integrations. **(a)** The model-agnostic Base pipeline uses plain-text anchors (`JSON:`) and a reduced `name (type)` parameter listing; the model continues past the leading JSON object with explanatory prose, so a strict whole-completion parse rejects the output (1.50 %). **(b)** B-template renders the request through Qwen 2.5's native chat template: the full schema is supplied inside `<tools>`, ChatML control tokens (`<|im_start|>`, `<|im_end|>`) mark turns, and the model emits its call inside `<tool_call>` tags, which the backend extracts (96.00 %). The two panels differ in prompt format and in how much of the source schema reaches the model; both are held fixed and replaced by a structural guarantee in CD.

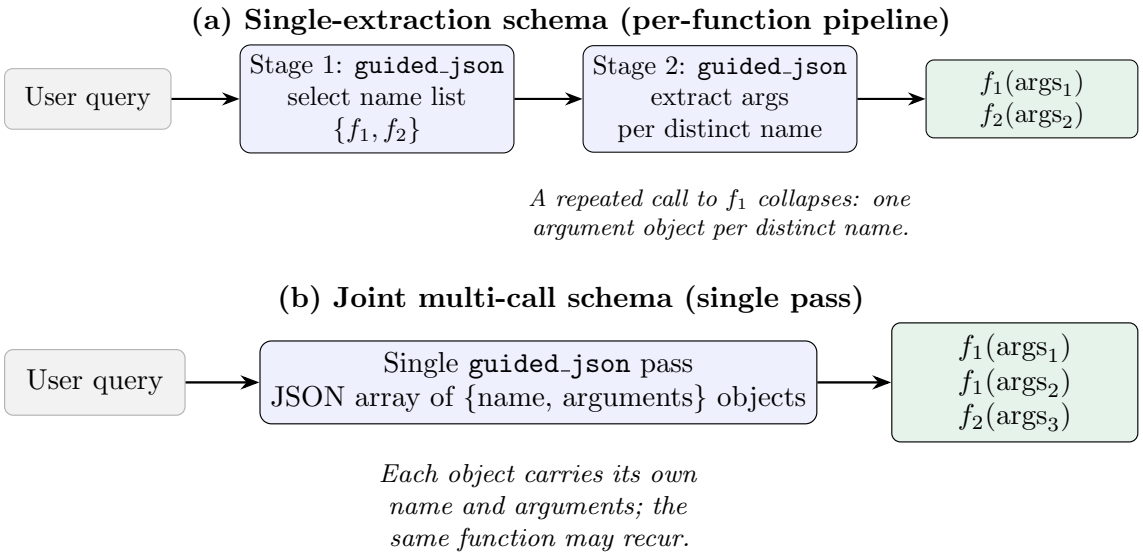

Figure 3.4: Single-extraction versus joint multi-call constrained-decoding schemas for the `parallel` categories. **(a)** The single-extraction pipeline first selects a list of distinct function names, then extracts an argument object for each name in a separate `guided_json` pass; because argument extraction is conditioned only on the query and the selected name and decoding is deterministic at temperature 0, two calls to the same function would be identical, so the design emits one object per distinct name. **(b)** The joint multi-call schema generates a single JSON array of complete call objects in one pass, so the same function can appear multiple times with different arguments. This is the change that lifts `parallel` from a structural 0.0% to 79.0% at 7B (Section 4.9).

(a) CD — template prompt (`name (type)` listing only)

```
Function: travel_itinerary_generator(destination: string, days: integer, daily_budget: integer,
    ↪ exploration_type: string) -> string
Description: Generate a travel itinerary based on specific destination, duration and daily budget,
    ↪ with preferred exploration type.
User request: Create an itinerary for a 7 days trip to Tokyo with daily budgets not exceeding $100
    ↪ and prefer exploring nature.
Extract the argument values as a JSON object with keys: destination (string), days (integer),
    ↪ daily_budget (integer), exploration_type (string).
JSON:
```

(b) CD+schema — enriched prompt (per-parameter schema block added)

```
Function: travel_itinerary_generator(destination: string, days: integer, daily_budget: integer,
    ↪ exploration_type: string) -> string
Description: Generate a travel itinerary based on specific destination, duration and daily budget,
    ↪ with preferred exploration type.
Parameters:
- destination (string, required): Destination city of the trip.
- days (integer, required): Number of days for the trip.
- daily_budget (integer, required): The maximum daily budget for the trip.
- exploration_type (string, optional): The preferred exploration type. Allowed values: "nature", "
    ↪ urban", "history", "culture". Default: "urban".
User request: Create an itinerary for a 7 days trip to Tokyo with daily budgets not exceeding $100
    ↪ and prefer exploring nature.
Extract the argument values as a JSON object with keys: destination (string), days (integer),
    ↪ daily_budget (integer), exploration_type (string).
JSON:
```

Figure 3.5: Template versus schema-enriched Stage 2 argument-extraction prompt for the same test case (`simple_python_34`). **(a)** The CD template prompt carries only the reduced `name (type)` parameter listing. **(b)** Config CD+schema inserts a `Parameters:` block with each parameter's description, enumerated values, and default, drawn directly from the BFCL function schema. The `guided_json` constraint is identical in both cases; only the prompt gains information. The enumerated values for `exploration_type`, which the template prompt omits, are exactly what the model needs to map "prefer exploring nature" onto the literal `"nature"`.

4 Results

This chapter presents the empirical results of the optimization ladder. It starts with the baseline and constrained decoding results, then compares the no-training, training-based, and schema-enriched configurations. The final sections broaden the evaluation to frontier models, multi-step agentic tasks, real-world GitHub MCP tool selection, model-size scaling, and harder BFCL categories.

4.1 Baseline Performance

Table 4.1 summarizes the baseline, the native-template control, and the constrained decoding result for Qwen 2.5 7B-Instruct on BFCL v4 `simple_python` (400 test cases). All other no-training configurations, including few-shot prompting, are reported in Section 4.2.

Table 4.1: BFCL v4 `simple_python` AST accuracy for Qwen 2.5 7B-Instruct: baseline (B), native-template control (B-template), and constrained decoding (CD).

Config	Description	Correct	Accuracy (↑)
B	Raw model, generic prompt, no optimization	6/400	1.5%
B-template	Native chat template, free generation	384/400	96.00%
CD	Constrained decoding (guided)	291/400	72.75%

Without constrained decoding, Base achieves only 1.5% accuracy. The dominant failure mode is format non-compliance: 394 out of 400 outputs fail to parse as valid JSON, producing `Extra data` errors or empty results. These failures do not show that the model cannot understand the queries. They show that the generic integration does not make the model express its answers in the required structured format.

The B-template control confirms this interpretation. When the same model is prompted through its native tool-calling template and allowed to generate freely, it reaches 96.00% (384/400). The 1.5% Base score is therefore a floor for the generic integration, not the model's capability ceiling. B-template also exceeds CD, which means the value of constrained decoding is not raw accuracy against a model-specific template. Its value is the structural guarantee: the same validity mechanism can be applied across models and schemas. Chapter 5 returns to this distinction.

Constrained decoding (CD) achieves 72.75% accuracy. Relative to the strict whole-completion baseline this is a 71.25 percentage point recovery; relative to the lenient first-object baseline measured in the isolation arm, the 7B same-parser gain is 10.75 pp. Function selection reaches 100% accuracy via `guided_choice`; in this category each test case provides a single candidate function, so guided selection cannot fail. The meaningful evidence on tool selection therefore comes from the `multiple` category (Section 4.9). The remaining 27.25% failure rate is entirely in argument extraction: wrong numeric precision (e.g., 9.8 vs. 9.81), string format mismatches (e.g., "kilometers/hour" vs. "km/h"), and incorrect optional parameter defaults.

The B-template control also frames the residual failure analysis in the remainder of this chapter. It characterises the model-agnostic pipeline rather than the model's ceiling. Unlike the pipeline's prompts, the native template carries the full unmodified function schemas. The failure categories that survive constrained decoding correspond closely to information

found in parameter descriptions, default values, and enumerations, which the pipeline omits (Chapter 3).

For a single frozen Qwen deployment, the native template is therefore the stronger local baseline. The constrained pipeline is valuable when the goal is a model-agnostic structural guarantee across heterogeneous models and schemas.

Few-shot prompting (PE) also uses constrained decoding and decreases accuracy by 2.5 percentage points relative to CD alone. The three examples target known failure modes, but they do not transfer across function domains. This negative result indicates that three in-context examples cannot substitute for the schema detail absent from the pipeline's prompts. Section 4.2.3 tests that point directly with CD+schema.

4.2 No-Training Methods

The following configurations modify only the inference pipeline; model weights are unchanged in all cases. CD establishes the structural baseline, while CD+schema tests whether restoring full schema detail can reduce the residual semantic error without training. PE, CD+Q+ITC, and CD+Q+RAG assess whether other inference-time interventions improve on the constrained pipeline.

4.2.1 Constrained Decoding

The two-stage constrained decoding pipeline eliminates the dominant failure mode of Base. In the function selection stage, `guided_choice` restricts generation to the set of available function names: function selection accuracy is 100% across all 400 test cases, confirmed by inspection of the raw prediction files. In the argument extraction stage, `guided_json` restricts generation to a JSON object conforming to the function schema, ensuring that every output is structurally valid [31, 16].

The 27.25% failure rate that remains after constrained decoding is entirely in argument values. Four categories account for the observed failures: wrong optional parameter defaults (e.g., `root_type=''` where `'all'` is expected), numeric precision errors (e.g., `9.8` where `9.81` is expected), string format mismatches (e.g., `"kilometers/hour"` where `"km/h"` is expected), and location format errors (e.g., `"New York"` where `"New York, NY"` is expected). These failures share a common structure: the model's output is semantically reasonable but does not match the specific convention that the benchmark's ground truth encodes. Constrained decoding enforces structural validity but cannot enforce semantic convention.

AWQ INT4 quantization reduces model memory from 14.25 GiB to 5.20 GiB (63.5% reduction) and inference startup time from 212 s to 35 s (83.5% reduction), with an accuracy change of -0.5 pp (289/400, 72.25%) [18]. The failure distribution does not change: no new error categories are introduced, and the relative frequency of existing failure types is unchanged. The accuracy difference of two test cases falls within run-to-run variance: six consecutive re-runs produced identical predictions (289/400), while re-runs later in the project period varied by one case (288/400). CD+Q therefore provides equivalent accuracy to CD at a substantially reduced memory footprint, and is the operating baseline for subsequent no-training experiments. Fig. 4.1 plots GPU memory against accuracy for the main configurations.

4.2.2 Prompt Engineering and Few-Shot

Three few-shot examples are prepended to the argument extraction prompt, constructed to target the most frequent failure categories observed in CD: numeric precision (`9.81` rather than `9.8`), string format with a state qualifier (`"San Francisco, CA"`), and Python

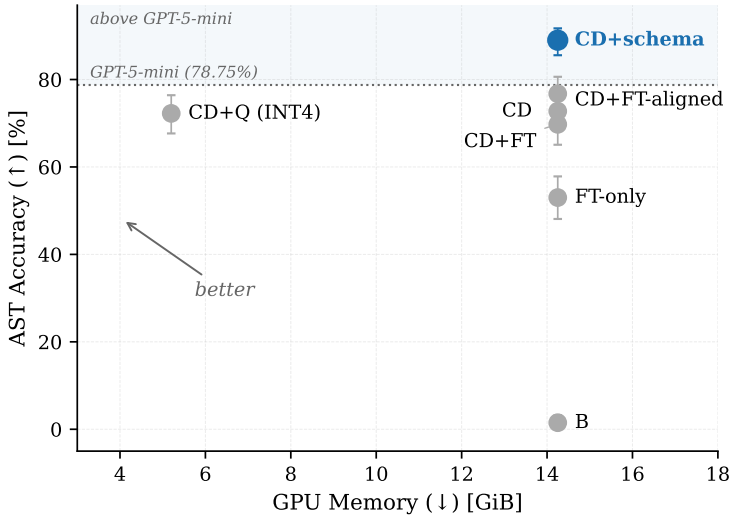

Figure 4.1: GPU memory footprint vs. BFCL `simple_python` AST accuracy for selected configurations. CD+Q achieves equivalent accuracy to CD at 63.5% lower memory. The dotted reference line marks the weakest frontier entry on this category (GPT-5-mini, 78.75%); the shaded band above it indicates frontier-level accuracy. Error bars are 95% binomial (Wilson) confidence intervals over the 400 test cases.

expression syntax (`x**2` rather than `x^2`). The result is 70.25% accuracy (281/400), a decrease of 2.5 pp relative to CD.

The failure categories from CD persist at similar or higher rates despite the targeted examples. The model does not transfer the demonstrated conventions to test cases from different function domains. Two mechanisms account for this outcome. First, the few-shot examples use function signatures from domains that differ from the test cases; the convention "use exact numeric precision" demonstrated for one function does not generalise to others. Second, the examples add approximately 300 tokens to each prompt without a proportional accuracy gain; for borderline cases where the base model produced a marginally correct output, the additional context appears to reduce accuracy slightly.

The result indicates that the argument extraction errors remaining after constrained decoding are not format gaps addressable by in-context examples [3]. The model's internal representation associates "acceleration due to gravity" with 9.8 and "fuel type" with "gasoline"; these priors are not shifted by example presentation alone.

4.2.3 Schema-Enriched Constrained Decoding

Config CD+schema enriches the Stage 2 argument extraction prompt with the full parameter schema detail: descriptions, default values, and enumeration constraints from the BFCL function definitions. All preceding configurations expose only parameter names and types.

CD+schema achieves 356/400 = 89.00% AST accuracy on `simple_python`, 16.25 pp above the canonical CD result (72.75%). Against the paired CD run co-evaluated on the same cases (72.50%), the gain is 16.50 pp; the 0.25 pp difference is one-case run-to-run drift between the two CD references. A paired McNemar test on stored per-case predictions yields Table 4.2.

The result is unambiguous: 73 vs. 7 discordant pairs cannot be produced by the ±0.5 pp run-to-run spread documented elsewhere in this chapter. The schema enrichment corrects cases whose failure after plain CD was attributable to missing value vocabulary: `fuel_type`

Table 4.2: Paired McNemar contingency for CD+schema vs. CD (7B, `simple_python`, 400 cases). b = CD wrong, CD+schema correct; c = CD+schema wrong, CD correct.

Outcome	Count
Both correct	283
Both wrong	37
CD wrong, CD+schema correct (b)	73
CD+schema wrong, CD correct (c)	7
$p < 0.00001$ (exact two-sided McNemar)	

parameters with an enum (`"gas"`, `"electric"`, `"hybrid"`) visible in the prompt, or `discount_rate` parameters whose description specifies a decimal fraction rather than a percentage.

The 44 remaining failures (11.00%) are predominantly value-strictness artifacts in the benchmark ground truth: `"Los Angeles"` where the scorer expects `"Los Angeles, CA"`, `1e-05` where it expects `0.0001`, or unit synonyms both valid in a real API. The model selects from the correct vocabulary; the AST equality check penalises the literal mismatch. These residual cases are not addressable by further schema enrichment.

Inference latency is not increased by schema enrichment. The mean per-call latency for CD+schema is 0.704 s (median 0.650 s, p95 1.086 s) across the full 400 sequential cases. Section 4.2.6 reports the full latency comparison across configurations.

4.2.4 Inference-Time Compute

A chain-of-thought reasoning step is inserted between function selection and argument extraction in CD+Q+ITC [30]. After the correct function is selected by `guided_choice`, the model generates a free-form reasoning trace of up to 256 tokens before the final `guided_json` extraction. The result is 65.5% accuracy (262/400), a decrease of 6.75 pp relative to CD+Q, and the lowest accuracy among all configurations that use guided decoding.

A flip analysis, computed by comparing per-case outcomes between CD+Q and CD+Q+ITC, separates the net change into individual gains and losses. Of the 400 test cases, chain-of-thought recovered 24 cases that were incorrect under CD+Q, while breaking 51 cases that were correct. The net change is -27 cases. Fig. 4.2 shows the four-way breakdown.

Table 4.3 illustrates four representative losses. In each case, CD+Q produces the value accepted by the benchmark, and the reasoning step overrides it with a linguistically natural alternative that the benchmark does not accept.

Three mechanisms account for this pattern. First, the reasoning step introduces drift: the model interprets the instruction to reason step by step as licence to produce derivations and justification paragraphs before the extraction step, consuming tokens without supplying useful information. Second, values committed in the reasoning text act as a prior for the subsequent guided extraction: once the reasoning block contains `discount_rate = 10`, the guided step inherits that value rather than re-evaluating it. Third, the model normalises values toward linguistically standard forms. The benchmark's ground truth encodes specific conventions, often arbitrary ones (abbreviations over full words, decimal fractions for percentages, particular date formats), that reasoning moves the model away from rather than toward. CD+Q's one-step guided decode produces the expected convention

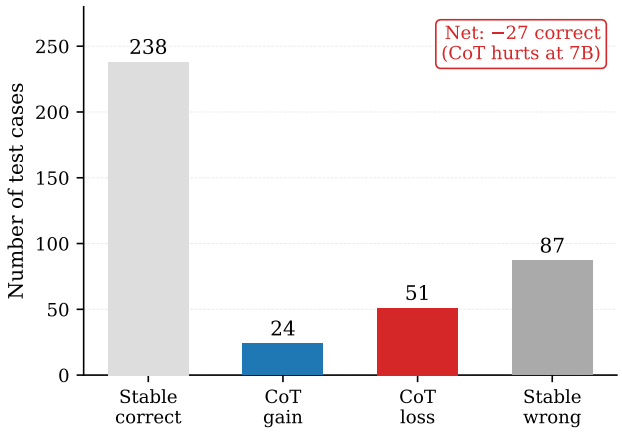

Figure 4.2: Flip analysis for CD+Q+ITC relative to CD+Q. Of 400 test cases, chain-of-thought breaks 51 previously correct predictions while recovering only 24 previously incorrect ones, for a net change of −27 cases.

Table 4.3: Representative cases where chain-of-thought reasoning overrides a correct CD+Q prediction. Reasoning excerpts are taken verbatim from stored reasoning traces.

Parameter	CD+Q	CoT	Reasoning excerpt
`discount_rate`	`0.1` ✓	`10.0` ×	"The discount rate is in percentage form. Extracted arguments: discount_rate = 10"
`fuel_type`	`'gas'` ✓	`'gasoline'` ×	"Gasoline would be the fuel type"
`duration`	`'2 weeks'` ✓	`'last 2 weeks'` ×	"The duration is 'last 2 weeks', which needs to be preserved exactly as a string"
`start_date`	`'01/01/2021'` ✓	`'2021-01-01'` ×	"start_date: '2021-01-01' (assuming ISO 8601 format)"

without verbalising it; verbalising the reasoning surfaces the model's preferred form, which consistently differs from the benchmark's.

The result extends the finding from PE: the residual failure rate after constrained decoding is caused by a mismatch between the model's priors and the benchmark's labels, and interventions that make reasoning explicit exacerbate this mismatch.

4.2.5 Retrieval-Augmented Generation

In all preceding configurations, each test case is provided with the exact function definition required to answer it. CD+Q+RAG removes this oracle assumption and replaces it with a retrieval step; the accuracy drop therefore measures the cost of candidate disambiguation, not a general failure of retrieval. All 370 unique function signatures from the `simple_python` category are embedded using `sentence-transformers/all-MiniLM-L6-v2` and indexed with FAISS [14]. At inference time, the query is embedded and the top-5 most similar functions are retrieved as candidates; the model must select the correct function from these candidates and extract its arguments [17]. The result is 47.75% accuracy (191/400), a decrease of 24.5 pp relative to CD+Q.

The retrieval component achieves recall@5 of 97.2% (389/400): the correct function is

present among the 5 retrieved candidates in 389 of 400 test cases. The 11 retrieval failures occur on domain-ambiguous functions whose names share little lexical overlap with the query (e.g., `calculate_density`, `recipe_search`, `sentiment_analysis`). Retrieval quality is therefore not the primary cause of the accuracy decrease.

The failure distribution identifies where accuracy is lost. Of the 400 test cases, 264 (66.0%) produce output identical to CD+Q, indicating that the additional candidates have no effect on these cases. In 128 cases (32.0%), the model selects a wrong function from the retrieved candidates. In 8 cases (2.0%), the correct function is selected but with incorrect argument values. The dominant failure mode is function disambiguation. When a query for triangle area retrieves `calculate_triangle_area`, `calculate_area`, and `calc_area_triangle` as candidates, the model selects a generic variant in the majority of cases. Similar patterns are observed for mathematical synonyms (`algebra.quadratic_roots` vs. `solve_quadratic_equation`) and namespace variants (`geometry.circumference` vs. `calculate_circumference`).

The result establishes that the bottleneck in a RAG-based tool-calling pipeline for a 7B model is not retrieval quality but candidate discrimination. A recall@5 of 97.2% is sufficient for the retrieval component; the model lacks the capacity to reliably distinguish among semantically similar candidate functions once they are retrieved. The 209 incorrect predictions fall into three groups: 128 with a wrong function selected, 8 with the correct function but wrong arguments, and 73 identical to CD+Q and wrong under both configurations. The last group is shared semantic failures that retrieval neither caused nor fixed. Fig. 4.3 summarises the outcome distribution over all 400 test cases.

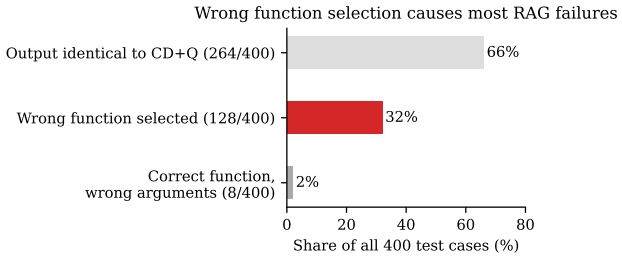

Figure 4.3: Outcome breakdown for CD+Q+RAG on the 7B model over all 400 test cases. The dominant RAG-specific failure mode is wrong function selection (32% of all cases), not retrieval miss or argument extraction error; 66% of outputs are identical to CD+Q.

4.2.6 Per-Call Inference Latency

Table 4.4 reports per-call inference latency for three configurations measured on the first 100 sequential `simple_python` test cases.

Table 4.4: Per-call inference latency for Qwen 2.5 7B-Instruct on BFCL v4 `simple_python` (first 100 cases, sequential dispatch, single L40S GPU, vLLM backend). AST accuracy is shown for context.

Config	Mean (s)	Median (s)	p95 (s)	Accuracy (↑)
B (no CD)	6.995	6.929	11.922	1%
CD	0.878	0.847	1.375	65%
CD+Q (AWQ)	0.661	0.623	1.006	63%

Constrained decoding is 8× faster than the unconstrained baseline (0.878 s vs. 6.995 s mean). The unconstrained model appends explanations, markdown formatting, or additional JSON

objects after the function call, generating far more tokens per request; the `Extra data` parse errors observed in Base are direct evidence of this behaviour. Constrained decoding terminates generation at the closing brace of a valid call, eliminating the surplus tokens. The per-step overhead of the logit mask does not reverse this advantage.

AWQ INT4 quantization reduces latency by a further 25% (0.661 s vs. 0.878 s mean; p95 1.006 s vs. 1.375 s). The two-case accuracy difference between CD (65/100) and CD+Q (63/100) in this subset is within the expected run-to-run variance; the first 100 cases contain a higher proportion of numeric-precision and enum-style cases than the full 400-case set.

Schema-enriched constrained decoding (Section 4.2.3) introduces no additional latency: the per-call mean across the full sequential 400-case run is 0.704 s, within the range observed for plain CD.

4.2.7 Technique Isolation Without Constrained Decoding

The configurations above apply each inference-time technique on top of constrained decoding, which measures whether the technique improves on the constrained-decoding ceiling. To isolate the contribution of constrained decoding itself, the three inference-time techniques are additionally evaluated without guided decoding, denoted PE-ng (few-shot), ITC-ng (chain-of-thought), and RAG-ng (retrieval), where *ng* indicates no guided decoding. These runs form the isolation arm of the experimental design rather than rungs on the eleven-configuration ladder, and are evaluated on the 7B model at full precision (bfloat16) on BFCL v4 `simple_python`. Table 4.5 reports the result.

Table 4.5: BFCL v4 `simple_python` AST accuracy for the inference-time techniques evaluated without constrained decoding (*ng*: no guided decoding; Qwen 2.5 7B-Instruct, bfloat16, 400 test cases). Config CD is shown for reference. All no-guided figures use the lenient first-complete-object parser, under which Config B scores 62.00% (vs. the 1.50% strict whole-completion floor; the two parse criteria are defined in Section 3.4).

Config	Description	Correct	Accuracy (↑)	Delta vs B (↑)
B	No technique, no CD	248/400	62.00%	baseline
RAG-ng	Retrieval, no CD	208/400	52.00%	−10.00 pp
PE-ng	Few-shot, no CD	236/400	59.00%	−3.00 pp
ITC-ng	Chain-of-thought, no CD	225/400	56.25%	−5.75 pp
CD	Constrained decoding	291/400	72.75%	+10.75 pp

Without constrained decoding, the no-guided base configuration (Config B) already reaches 62.00% under the corrected parser, and none of the three prompt-level techniques exceeds it: few-shot (PE-ng) reaches 59.00% (236/400, −3.00 pp), chain-of-thought (ITC-ng) 56.25% (225/400, −5.75 pp), and retrieval (RAG-ng) 52.00% (208/400, −10.00 pp). Constrained decoding (72.75%) is the only configuration that exceeds the base, by +10.75 pp.

The isolation measurements also quantify the marginal contribution of constrained decoding on top of each technique. Table 4.6 reports the accuracy of each technique with and without guided decoding, together with the resulting constrained-decoding contribution.

Once a lenient first-object parser is in place, the marginal contribution of constrained decoding on the 7B model is modest: +10.75 pp over the base, +11.25 pp on top of few-shot, and +9.25 pp on top of chain-of-thought. For retrieval it is negative (−4.25 pp): RAG-ng (52.00%) exceeds CD+Q+RAG (47.75%). The value of constrained decoding therefore lies in its structural format guarantee, which recovers the strict-parse baseline

Table 4.6: Marginal contribution of constrained decoding for each inference-time technique on the 7B model. The guided chain-of-thought and retrieval figures are measured with AWQ INT4 quantization (CD+Q+ITC and CD+Q+RAG); the 0.5 pp quantization effect at 7B is small relative to the constrained-decoding contribution.

Technique	Without CD (↑)	With CD (↑)	CD contribution
None (CD alone)	62.00%	72.75%	+10.75 pp
Few-shot	59.00%	70.25%	+11.25 pp
Chain-of-thought	56.25%	65.50%	+9.25 pp
Retrieval	52.00%	47.75%	−4.25 pp

from 1.50% to a well-formed output in every case, rather than in a large accuracy margin over prompt-level techniques.

4.3 Training-Based Methods

4.3.1 LoRA Fine-Tuning

A LoRA adapter is trained on the Salesforce `xlam-function-calling-60k` dataset [35], using 54,000 training examples and 6,000 validation examples. The base model is `Qwen/Qwen2.5-7B-Instruct`, trained for 2 epochs with LoRA rank 16 in bfloat16 [12]. The adapter is merged into the base weights by linear interpolation before evaluation, producing a single merged model at full bfloat16 precision.

Without constrained decoding (FT-only), the fine-tuned model reaches 53.00% accuracy (212/400) under the corrected lenient parser, below the no-guided base configuration (62.00%) measured the same way. Plain LoRA fine-tuning without constrained decoding therefore does not exceed the unoptimised base on format compliance for this category.

The xlam dataset encodes assistant-turn responses as Python call syntax (`func(arg=value, ...)`). This convention differs from the evaluation pipeline, which expects a JSON argument object (`{"arg": value}`). The format mismatch between the training distribution and the evaluation format is the primary limitation of this configuration, and its effect on the combined pipeline is examined in Section 4.4.

4.4 Combined Configurations

CD+FT combines constrained decoding with the LoRA-merged model. The result is 69.75% accuracy (279/400), a regression of 3 pp relative to CD without fine-tuning. The combined configuration underperforms the no-fine-tuning baseline.

The regression is attributed to the training format mismatch. Constrained decoding enforces the surface format: every output is a valid JSON object conforming to the function schema. However, the argument value predictions remain biased toward the conventions of the training data. Since xlam encodes argument values as part of Python call syntax, the fine-tuned model's weights encode this convention. When constrained decoding forces the output to be valid JSON, the value-level predictions are pulled toward xlam conventions rather than the BFCL ground truth. A secondary contributing factor is the composition of the training set: 52.6% of xlam examples contain multiple function calls per query, while the `simple_python` category is exclusively single-call. This distribution mismatch may shift the model's prior toward multi-call patterns in contexts where a single call is required.

The gap between FT-only (53.00%) and CD+FT (69.75%) shows that constrained decoding still adds a clear margin even after fine-tuning: removing it reduces accuracy by 16.75 pp.

The two techniques are complementary, but fine-tuning on a misaligned training distribution does not improve on the CD baseline because the learned value conventions do not match those of the evaluation benchmark.

4.4.1 Format-Aligned Fine-Tuning

The format mismatch identified in CD+FT is addressed by rewriting the training data preprocessor to produce assistant-turn responses that match the inference pipeline's argument extraction prompt exactly. Two variants are evaluated: FT-aligned-ng (*ng*: no guided decoding), which applies the format-aligned adapter without constrained decoding, and CD+FT-aligned, which combines the adapter with the full constrained decoding pipeline.

FT-aligned-ng achieves 41.00% accuracy (164/400) under the corrected lenient parser, a decrease of 12.00 pp relative to FT-only (53.00%). The format-aligned training objective optimises for JSON-only argument extraction and omits the function name from the assistant turn, as required by the guided inference pipeline. The unguided evaluator must recover the function name from the raw output; because the training format no longer produces it, the unguided pass rate decreases. Format alignment improves the guided inference path at the cost of degrading the unguided path.

This degradation generalises across model families. Evaluated without guided decoding on the same harness, the format-aligned adapters fall to 0.00%–3.75% accuracy (Phi-4-mini 0/400; Llama-3.2-3B 9/400; Llama-3.2-1B 12/400; gemma-3-1b 15/400), whereas the matched instruct checkpoints retain 39.00%–67.50% under the same no-guided path (Phi-4-mini 270/400; Llama-3.2-3B 233/400; Llama-3.2-1B 165/400; gemma-3-1b 156/400). The drop is steeper than the 7B Qwen point (62.00%→41.00%) and largest for Phi-4-mini, whose aligned model emits an empty completion for every case. Format alignment specialises the model to the constrained-decoding protocol: its standalone accuracy falls strictly below the unmodified instruct checkpoint, increasingly so away from the 7B Qwen setting.

The training loss for both adapters is shown in Appendix B (Fig. B.1). Both runs use identical hyperparameters and converge to similar final losses (v1: 0.203, v2: 0.229 at step 6750), confirming that the v1 regression is not a training failure. The difference in BFCL accuracy (69.75% vs. 76.75%) is attributable to the training format, not to convergence quality.

CD+FT-aligned achieves 76.75% accuracy (307/400), an improvement of 4.0 pp over CD and 7.0 pp over CD+FT. This is the strongest training-based configuration and the only fine-tuning configuration to exceed the no-training CD level of 72.75%. Significance is assessed with McNemar's exact test over the per-case predictions, since both configurations use the same 400 cases. On the stored paired files, fine-tuning corrects 45 cases and breaks 27 ($p = 0.044$).[1]

This result is nominally below the 0.05 threshold but lies within the observed run-to-run spread: a CD run at the upper end of the 289–291 range would not reach significance. The p-value is also not corrected for the number of comparisons in the full ablation grid. The single-category delta is therefore treated as borderline rather than established. The stronger evidence for format alignment is the consistency of the effect across scale: the gain is positive at all four model sizes (Section 4.8).

[1]The CD reference scores 289/400 in the stored paired file; repeated CD runs varied between 289 and 291 of 400 over the project period (Section 3). The single-proportion confidence interval reported in Section 4.5 applies to absolute accuracies, not to paired differences.

The 23.25% failure rate that remains under CD+FT-aligned follows the same taxonomy as CD: optional parameter defaults, numeric precision, string format conventions, and enum value mismatches. The xlam training examples do not supply argument values that match BFCL ground truth labels closely enough to eliminate these failure categories. The 4.0 pp gain over CD reflects a reduction in error frequency, not the introduction of a new success mechanism.

4.4.2 Quantized Fine-Tuning

Config CD+Q+FT-aligned applies AWQ INT4 quantization to the format-aligned LoRA-merged model. The research question is whether the 4.0 pp gain from CD+FT-aligned survives quantization.

CD+Q+FT-aligned achieves 74.25% accuracy (297/400), an improvement of 1.5 pp over Config CD and 2.0 pp over Config CD+Q. The quantization penalty relative to the unquantized CD+FT-aligned is 2.5 pp (76.75% → 74.25%).

This penalty is larger than the 0.5 pp loss observed when quantizing the base model (Config CD → Config CD+Q), suggesting that AWQ interacts more destructively with fine-tuned weight deltas than with the base weights. A plausible explanation is that the AWQ calibration set is a general-purpose text corpus: the fine-tuned weights encode function-calling conventions that are underrepresented in the calibration distribution, leading to suboptimal rounding of the LoRA-induced weight changes.

Despite the larger quantization penalty, the full stack retains a positive return on fine-tuning: CD+Q+FT-aligned (74.25%) outperforms both the no-training ceiling Config CD (72.75%) and the quantized baseline Config CD+Q (72.25%). McNemar's exact test does not confirm this gain. On the stored paired predictions, CD+Q+FT-aligned corrects 46 cases and breaks 37 ($p = 0.38$), within discordant-pair noise.[2] Unlike the full-precision comparison, the advantage over CD+Q is therefore directional rather than statistically established. From a deployment perspective, CD+Q+FT-aligned occupies the same memory footprint as Config CD+Q (approximately 5.2 GiB of GPU memory), making fine-tuned capability available on consumer hardware at INT4 size.

Table 4.7 presents all twelve configurations ordered by accuracy. Among the training-based configurations, only CD+FT-aligned and CD+Q+FT-aligned exceed Config CD; the strongest configurations are placed against frontier models in the next section (Fig. 4.4).

4.5 Comparison with Frontier Models

Table 4.8 compares the Qwen 2.5 7B-Instruct results against frontier models on BFCL v4 Python Simple AST. The frontier scores are official leaderboard scores rather than local re-runs.[3]

Fig. 4.4 and Table 4.8 compare the Qwen 2.5 7B results against the leaderboard. The native-template control (B-template, 96.00%) lies within the band spanned by flagship frontier models on this category (94.75%–97.75%), consistent with the 94.50% reported for the same model under the official harness on an earlier benchmark revision [4]: the

[2] The CD+Q reference scores 288/400 in the stored file, within the documented 288–289 run-to-run spread.

[3] https://gorilla.cs.berkeley.edu/leaderboard.html, accessed 2026-06-04 and re-verified 2026-06-10 (the page reports its data as last updated 2026-04-12). The quoted scores are the Python sub-category of the Non-Live Simple AST group (400 test cases, identical in composition to the `simple_python` category evaluated here), taken from the leaderboard's published data file. The "Simple" column displayed on the leaderboard page is the unweighted mean over the Python (400 cases), Java (100 cases), and JavaScript (50 cases) sub-categories and is therefore not comparable to a Python-only evaluation.

Table 4.7: BFCL v4 `simple_python` AST accuracy across all twelve configurations. Δ is relative to Config CD (72.75%). Config B is the strict whole-completion-parse floor; the no-CD figures FT-only and FT-aligned-ng use the corrected lenient first-object parser (Table 4.5).

Config	Technique	Accuracy (↑)	Δ vs CD
B	Raw baseline	1.50%	−71.25 pp
FT-aligned-ng	Format-aligned LoRA, no CD	41.00%	−31.75 pp
CD+Q+RAG	CD + AWQ + RAG top-5	47.75%	−25.00 pp
FT-only	LoRA, no CD	53.00%	−19.75 pp
CD+Q+ITC	CD + AWQ + CoT	65.50%	−7.25 pp
CD+FT	CD + LoRA (misaligned)	69.75%	−3.00 pp
PE	CD + few-shot	70.25%	−2.50 pp
CD+Q	CD + AWQ INT4	72.25%	−0.50 pp
CD	Constrained decoding	72.75%	–
CD+Q+FT-aligned	CD + AWQ INT4 + format-aligned LoRA	74.25%	+1.50 pp
CD+FT-aligned	CD + format-aligned LoRA	76.75%	+4.00 pp
CD+schema	**CD + schema enrichment (7B)**	**89.00%**	**+16.25** pp

model itself performs at frontier level when prompted through its native template. Plain CD (72.75%) and CD+FT-aligned (76.75%) sit below the frontier range on this category; CD+schema (89.00%), which adds full parameter schema detail to the constrained prompt, falls within the lower BFCL v4 Python Simple AST leaderboard band, between GPT-5-mini (78.75%) and GPT-4.1 (91.00%). The gap to the best frontier model (Claude Sonnet 4.5 at 97.75%) is 8.75 pp.

Constrained decoding closes the integration gap, not the frontier gap, for a prompt that omits schema detail. Restoring that detail (parameter descriptions, enumeration values, and defaults, which the native template and frontier APIs provide) closes most of the residual gap without training. The comparison with B-template bounds the remaining gap at 7.00 pp above CD+schema. Because B-template changes prompt format, schema completeness, tool-call delimiters, and parser target together, this control does not by itself identify which factor accounts for the gap; the remaining difference is consistent with a combination of native-template prompt-format advantage and residual value-convention scorer artifacts.

A note on statistical precision: with $n = 400$ binary test cases, the 95% Wilson confidence interval is approximately ±4.1–4.4 pp for accuracy values near 72–77%. The difference between CD+FT-aligned (76.75%) and GPT-5-mini (78.75%) is therefore within the confidence interval of a single evaluation run, and is best read as parity with the weakest frontier entry rather than a definitive ranking. The 14 pp or larger gaps to the flagship models are well outside it.

A second caveat concerns the harness. Frontier model scores are taken from the official BFCL leaderboard and are not re-evaluated locally, so methodology differences may affect comparability at the margin. The B-template control bounds this concern: 96.00% under the local harness falls inside the 94.75%–97.75% flagship band on the same category, indicating that the local evaluation and the leaderboard are commensurable at the category level.

Cross-harness comparisons in this section therefore remain indicative. The internally

Table 4.8: BFCL v4 `simple_python` AST accuracy: Qwen 2.5 7B configurations vs. frontier models (official leaderboard scores, Python Simple AST category).

Model	Type	Accuracy (↑)
Claude Sonnet 4.5	Frontier	97.75%
Claude Opus 4.5	Frontier	96.50%
Claude Haiku 4.5	Frontier	95.00%
Gemini 3 Pro	Frontier	94.75%
GPT-4.1	Frontier	91.00%
GPT-4.1-mini	Frontier	91.00%
GPT-5-mini	Frontier	78.75%
Qwen 2.5 7B B-template (control)	SLM (ours)	96.00%
Qwen 2.5 7B + CD+schema	**SLM (ours)**	**89.00%**
Qwen 2.5 7B + CD+FT-aligned	SLM (ours)	76.75%
Qwen 2.5 7B + CD	SLM (ours)	72.75%
Qwen 2.5 7B + PE	SLM (ours)	70.25%
Qwen 2.5 7B (raw)	SLM (ours)	1.50%

consistent comparisons are those between configurations evaluated under the same local harness.

The frontier comparison also requires careful scope. The `simple_python` category is the least demanding BFCL scenario: each test case is single-turn, uses one function call, and provides one candidate function. Both constrained decoding and frontier models' native tool-calling APIs guarantee valid JSON and valid function names, so format compliance is largely equalized. The differentiator is argument value accuracy. On that dimension, flagship frontier models hold a 14–21 pp advantage over the best constrained configuration, while the same 7B model under its native template is competitive with them. The phrase "frontier range" in this section therefore refers only to BFCL v4 Python Simple AST, not to general agentic performance.

Frontier models distinguish themselves on harder tasks. Their BFCL overall scores, which weight multi-turn (30%) and agentic tasks (40%), remain substantially higher than what an SLM would likely achieve on those categories. Claude Opus 4.5 maintains 77.47% overall, while GPT-4.1 drops to 53.96%, indicating that even among frontier models, simple function calling does not predict general agentic capability. The multi-step evaluation in Section 4.6 quantifies where the SLM–frontier gap re-emerges.

4.6 Agentic Evaluation

Single-call tool accuracy measures one dimension of agent capability. The τ-bench retail benchmark evaluates end-to-end task completion across multi-turn interactions, where the model must select tools, observe results, and iterate until a retail service task is resolved [33]. CD is evaluated on the retail domain test split (115 tasks) to establish the agentic baseline.

Three full runs are completed to account for user simulator variance. Pass rates of 4.35%, 5.22%, and 3.48% are observed for runs 1, 2, and 3 respectively, with a mean of 4.35% (5/115). The user simulator (Qwen 2.5 7B on the same vLLM instance) generates stochastic responses despite the model running at temperature 0.0, producing the observed inter-run variance. At $n = 115$ binary tasks the standard error is approximately 1.9 pp; the observed range of 3.5–5.2% is within this noise band.

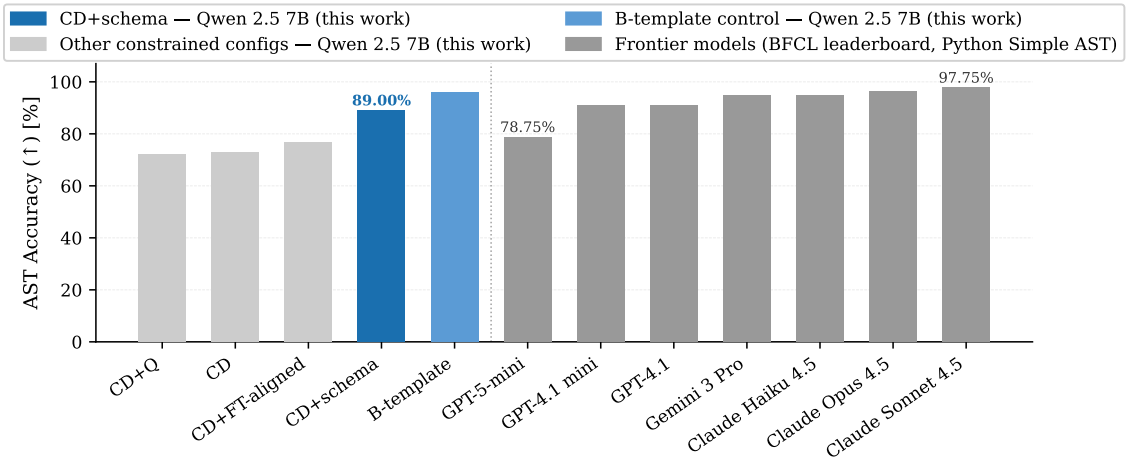

Figure 4.4: BFCL v4 `simple_python` AST accuracy for Qwen 2.5 7B configurations (this work) vs. frontier models from the official leaderboard (Python Simple AST category). The native-template control (B-template, 96.00%) lies within the flagship frontier band; CD+schema (89.00%) falls within the lower BFCL Python Simple AST leaderboard band.

Scoring is binary: a task passes only if the final database state exactly matches the ground truth at the end of the interaction. Partial completion yields reward 0.0. This criterion reflects the operational requirement that retail service tasks must be resolved completely; partial execution has no customer-facing value.

Both the agent and the user simulator are 7B models in this setup, whereas the reference τ-bench configuration drives the simulator with a frontier model. A 7B simulator can misstate or omit task details, so part of the observed failure rate may originate in the simulator rather than the agent. The absolute pass rate should therefore be read as a non-standard estimate, and comparisons to published τ-bench pass rates obtained under stronger simulators are not like-for-like (Section 5.3.1).

Table 4.9 places the single-call and multi-step results side by side.

Table 4.9: Single-call accuracy (BFCL) vs. multi-step pass rate (τ-bench) for CD on Qwen 2.5 7B-Instruct.

Benchmark	Setting	Pass rate (↑)
BFCL `simple_python`	Single call, 1 turn	72.75%
τ-bench retail	Multi-step, 5–15 turns	4.35%

The 68.4 pp gap between the two benchmarks is the central result of this section. A model that generates individually valid tool calls 72.75% of the time achieves end-to-end task success only 4.35% of the time on the retail domain.

Four failure categories are identified by trajectory inspection. State drift is the most frequent: the model does not track which items have been modified across turns when a request involves multiple sub-tasks. Premature termination is observed when the model calls `finish` before all sub-tasks are resolved, particularly on multi-item requests. Tool parameter errors persist despite CD achieving near-perfect function selection on BFCL, because τ-bench tools accept free-form string identifiers that constrained decoding does not validate. Context management failures are also observed in tasks with long conversation

histories, where earlier tool call results are displaced from the model's working context.

Figure 4.5 shows a representative failed trajectory that makes the gap concrete. The task requires the agent to authenticate Yusuf Rossi (zip 19122) and exchange two items in order `#W2378156`. The policy in the system prompt requires the agent to locate the user id first, by email or by name and zip. Instead of asking for these details or using the identity available in the conversation, the model fabricates them: it calls `find_user_id_by_email` with an invented address (`john.doe@example.com`), and after that fails, calls `find_user_id_by_name_zip` with an invented name and zip (`John Doe`, `10001`). Both lookups return `user not found`, and the agent transfers to a human, so none of the requested exchanges is performed and the task scores reward 0.0. Every tool call in the trajectory is structurally valid and names a real function, which is exactly what constrained decoding guarantees; the failure is in the *values*. The free-form string identifiers that `find_user_id_by_email` and `find_user_id_by_name_zip` accept are not constrained by `guided_json`, so the model is free to hallucinate them, and a single fabricated argument at the authentication step collapses the entire multi-step task before any sub-goal is reached. This is the BFCL residual failure mode (Section 4.2.1) acting at the first turn of a multi-step trajectory, where there is no later step that can recover from it.

```
Task: authenticate Yusuf Rossi (zip 19122); exchange two items in #W2378156.
Ground-truth first action:
  find_user_id_by_name_zip(first_name="Yusuf", last_name="Rossi", zip="19122")

[user]      I received my order #W2378156, and I'm looking to exchange the
            mechanical keyboard [...] switch the smart thermostat [...]
[assistant] -> find_user_id_by_email({"email": "john.doe@example.com"})
[tool]      Error: user not found
[assistant] -> find_user_id_by_name_zip({"first_name": "John",
                                         "last_name": "Doe", "zip": "10001"})
[tool]      Error: user not found
[assistant] -> transfer_to_human_agents({"summary": "User John Doe could not
            be located [...]"})
[tool]      Transfer successful        (task ends; reward 0.0)
```

Figure 4.5: A failed τ-bench retail trajectory for CD on Qwen 2.5 7B-Instruct (task 0, run 1), reproduced from the stored trajectory log. Each tool call is structurally valid and names an existing function, the guarantee constrained decoding provides, but the model fabricates the free-form identifier arguments (`john.doe@example.com`, then `John Doe`/`10001`) instead of eliciting or reusing the user's actual identity. Authentication fails at the first step, so the multi-step task collapses before any exchange is attempted. Assistant prose between calls is omitted; `[...]` marks elisions.

The 4.35% pass rate on τ-bench establishes the agentic baseline for CD on the retail domain. The 68.4 pp gap quantifies the boundary separating single-call function calling from reliable multi-step agentic task completion.

4.6.1 Size Sweep

Table 4.10 reports τ-bench retail pass rates for CD and CD+FT-aligned across all four Qwen 2.5 model sizes.

Pass rates are uniformly low across all model sizes. All conditions except 7B CD are single runs; at $n = 115$ binary tasks, one task corresponds to 0.87 pp and the standard error is approximately 1.9 pp. The full spread across all conditions (1.74%–4.35%) is less than two standard errors, so no ranking between individual conditions is supported. These results

Table 4.10: τ-bench retail domain pass rate across Qwen 2.5 model sizes (115 tasks per condition; 7B CD is the mean of three independent runs, all other conditions are single runs). The full spread (1.74%–4.35%) is within two standard errors (SE $\approx$ 1.9 pp at n=115); no ranking between individual conditions is statistically supported.

Size	CD	CD+FT-aligned
0.5B	3.48% (4/115)	0.00% (0/115)
1.5B	1.74% (2/115)	3.48% (4/115)
3B	4.35% (5/115)	4.35% (5/115)
7B	4.35% (mean, 3 runs)	3.48% (4/115)

establish a shared performance floor, not a scale trend. This contrasts with the BFCL `simple_python` sweep, where CD accuracy increases by 21.25 pp from 0.5B to 7B. Scale improves single-call argument extraction but does not improve multi-turn task completion.

CD+FT-aligned does not improve τ-bench pass rates relative to CD despite producing BFCL gains at every model size (Section 4.8). At 0.5B, CD+FT-aligned collapses to 0.00%, a decrease of 3.48 pp relative to the CD baseline at the same size. At 1.5B and 3B the pass rates are equal within noise; at 7B the mean of 4.35% for CD lies within the single-run standard error of the 3.48% CD+FT-aligned result. Format-aligned fine-tuning, which increases BFCL `simple_python` accuracy by up to 7.75 pp, does not transfer to multi-turn task completion.

4.7 Real-World Tool Selection

The BFCL and τ-bench evaluations isolate two aspects of tool calling under controlled conditions. The `simple_python` category presents a single oracle function per test case, and τ-bench provides a fixed, known tool catalogue. Neither reproduces the conditions of a live deployment, where the model must select among a full catalogue of semantically similar operations and then produce valid arguments against a real API. This section reports a targeted evaluation under those conditions using the GitHub Model Context Protocol (MCP) server: 21 real GitHub tools and 50 hand-written natural-language prompts, of which the 11 read-only tools are executed live against the GitHub REST API. The evaluation is deliberately narrow—a single domain with hand-crafted prompts—and is intended as a real-world validation rather than a replacement for BFCL. A secondary question is whether the retrieval disambiguation collapse observed in Config CD+Q+RAG ($-$24.5 pp as the candidate set grew, Section 4.2) reappears at the scale of a real 21-tool catalogue.

As with the BFCL baseline (Section 4.1), the no-guided baseline (Config B) is sensitive to the output parser; the parser used here was hardened during the project to recover tool calls from free-form completions. All configurations in Table 4.11 are scored under this single hardened parser, so the baseline and constrained-decoding rows share one measurement basis.

The baseline result is load-bearing in the same way as the BFCL B-template control. Config B reaches 92.0% tool selection because the hardened parser recovers function calls from the model's free-form output; the baseline therefore leans on parser engineering much as the BFCL B-template leans on the model's native chat template (Section 4.1). Against this baseline, constrained decoding adds only 4 pp of tool selection (92.0% to 96.0%) and leaves argument accuracy unchanged at 80.0%. As on `simple_python`, the value of

Table 4.11: GitHub MCP real-world evaluation, Qwen 2.5 7B-Instruct (50 natural-language prompts over a 21-tool catalog). All configurations are scored under the hardened no-guided parser; constrained-decoding outputs parse unconditionally, so the baseline and constrained-decoding rows share one measurement basis.

Config	Tool selection (↑)	Arg accuracy (↑)	Exec. success (read-only)
B (baseline)	92.0% (46/50)	80.0% (40/50)	93.3% (28/30)
CD	96.0% (48/50)	80.0% (40/50)	90.0% (27/30)
CD+Q (AWQ 4-bit)	98.0% (49/50)	80.0% (40/50)	86.2% (25/29)

constrained decoding over an already-capable model is the structural guarantee that every output is a valid call, not a large accuracy margin. Quantization is again close to free: Config CD+Q reaches 98.0% tool selection and 80.0% argument accuracy at roughly half the memory and a faster load, and its outputs parse unconditionally. Live execution of the read-only tools succeeds on 86–93% of calls (HTTP status below 400), confirming that these are real tool invocations rather than label matches. The constrained-decoding execution failures are all `search_issues` calls returning HTTP 422, where the model supplies a natural-language phrase instead of GitHub search syntax—a gap in model knowledge that `guided_json` cannot close.

Table 4.12: GitHub MCP tool selection and argument accuracy, baseline (B) versus constrained decoding (CD), across model families (50 prompts each, single hardened parser). The accuracy benefit of constrained decoding concentrates on the weakest viable model; for models that already comply it is marginal or negative.

Model	B tool	CD tool	B arg	CD arg
Gemma-3 1B-Instruct	36%	38%	24%	22%
Llama-3.2 1B-Instruct	46%	82%	34%	56%
Llama-3.2 3B-Instruct	94%	80%	74%	60%
Phi-4-mini-Instruct	88%	90%	78%	78%
Qwen 2.5 7B-Instruct	92%	96%	80%	80%

Across model families the benefit of constrained decoding concentrates on the weakest viable model. Llama-3.2 1B gains 36 pp of tool selection (46% to 82%), whereas for models that already comply the effect is marginal or negative: Phi-4-mini gains 2 pp, Qwen 2.5 7B gains 4 pp, and Llama-3.2 3B loses 14 pp (94% to 80%). Gemma-3 1B sits below the viability floor—36% baseline, 38% constrained, with argument accuracy near 22%—and no configuration recovers it. This pattern confirms the framing established on `simple_python` (Section 4.1): the value of constrained decoding is the structural guarantee, not added accuracy over a capable model. The Llama-3.2 3B regression has an identified mechanism, validated under identical baseline and constrained prompts: `guided_choice` over 21 candidate tools shifts the 3B model toward the broad `search_issues` tool, which accounts for 7 of its 9 new errors. Forcing a choice over a large catalogue therefore carries a real disambiguation cost for smaller models.

This result refines the retrieval finding of Section 4.2. Config CD+Q+RAG lost accuracy as the candidate set grew, but constrained decoding over the full 21-tool catalogue holds 96.0% tool selection on Qwen 2.5 7B, indicating that the earlier collapse was driven by retrieval noise rather than by tool count as such. The Llama-3.2 3B regression qualifies

this: large catalogues do impose a disambiguation cost on smaller models, so the conclusion is a nuanced one rather than a flat claim that tool count is free.

This evaluation is limited in scope and complements the threats to validity in Section 5.3.1. It covers 50 hand-crafted prompts in a single domain; the write tools are scored on selection and argument accuracy only and are not executed; and the pipeline tests function selection and argument construction rather than end-to-end MCP transport, since the predicted function and arguments are translated into a REST call directly. As elsewhere, the baseline is parser-sensitive, so the baseline-to-constrained comparison should be read under the single hardened parser used throughout this section.

4.8 Model-Size Scaling

Table 4.13 reports BFCL v4 `simple_python` accuracy for all seven configurations across all four Qwen 2.5 Instruct sizes.

Table 4.13: BFCL v4 `simple_python` AST accuracy across all configurations and Qwen 2.5 model sizes (400 test cases each). The Config B row reports the strict whole-completion-parse floor (Section 3.4); only the 7B value (1.50%) was independently re-grounded under strict parsing (the smaller-size values are the near-zero pre-fix measurements and approximate that floor).

Config	0.5B	1.5B	3B	7B
B	3.50%	4.75%	2.75%	1.50%
B-template	78.00%	84.75%	95.00%	96.00%
CD	51.50%	62.25%	64.75%	72.75%
CD+schema	74.25%	84.00%	88.00%	89.00%
CD+Q	47.75%	60.50%	64.50%	72.25%
PE	53.50%	64.75%	67.00%	70.25%
CD+Q+ITC	42.00%	54.75%	58.00%	65.50%
CD+Q+RAG	26.00%	34.75%	48.25%	47.75%
CD+FT-aligned	59.25%	66.00%	66.75%	76.75%

Base accuracy is near-zero across all model sizes, ranging from 1.50% to 4.75%, with no consistent trend with scale. The 3B model achieves 2.75%, lower than both the 1.5B (4.75%) and 0.5B (3.50%) models; at this accuracy range, the absolute difference is at most 8 correct outputs out of 400, which is within the noise floor of near-zero unguided generation. The non-monotonic ordering across sizes is not statistically meaningful and does not reflect a capacity difference. What the near-zero baseline uniformly confirms is that format non-compliance dominates at every scale: increasing model size alone does not recover structured output generation without an explicit constraint mechanism.

CD accuracy scales consistently with model size. The 0.5B model achieves 51.50%, the 1.5B achieves 62.25%, the 3B achieves 64.75%, and the 7B achieves 72.75%. The CD gain over Base also increases with model size, from 48.0 pp at 0.5B to 71.25 pp at 7B. Larger models produce fewer argument value errors once format compliance is enforced, and a larger fraction of their latent capability is unlocked by the constraint mechanism. No size reaches the BFCL Python Simple AST leaderboard band on this category (78.75%–97.75%); the 7B model under CD comes within 6.00 pp of the weakest listed frontier entry (GPT-5-mini, 78.75%).

AWQ INT4 quantization applied to CD produces a size-dependent accuracy cost. At

0.5B the penalty is 3.75 pp (51.50% → 47.75%); at 1.5B it falls to 1.75 pp; at 3B and 7B it is 0.25 pp and 0.50 pp respectively, within run-to-run variance. Quantization cost is negligible above 1.5B, confirming that INT4 quantization is a safe operating point at 3B and 7B. At 0.5B the penalty is meaningful but remains acceptable relative to the 4× memory reduction.

Few-shot prompting (PE) produces a size-dependent effect that reverses at 7B. At 0.5B, 1.5B, and 3B, PE improves accuracy over CD by 2.00, 2.50, and 2.25 pp respectively. At 7B, PE decreases accuracy by 2.50 pp (70.25% vs. 72.75%). At smaller model sizes the prepended examples appear to act as format anchors, providing a marginal improvement. At 7B, the additional context length reduces accuracy on borderline cases. PE is not a reliable technique across model sizes: the direction of effect depends on scale.

Chain-of-thought reasoning (CD+Q+ITC) reduces accuracy at every model size. The decrease relative to CD+Q is 5.75 pp at 0.5B, 5.75 pp at 1.5B, 6.50 pp at 3B, and 6.75 pp at 7B. The penalty is approximately constant across the full size range. Chain-of-thought is not a capability that smaller models lack and larger models can exploit: the mechanism that causes the accuracy reduction operates regardless of model scale.

Retrieval-augmented generation (CD+Q+RAG) reduces accuracy at every model size. The decreases relative to CD+Q are 21.75 pp at 0.5B, 25.75 pp at 1.5B, 16.25 pp at 3B, and 24.50 pp at 7B. The pattern is non-monotonic: the 3B model loses less than the 1.5B and 7B models. As retrieval recall@5 is 97.2%, retrieval quality is not the limiting factor. The dominant failure mode is candidate disambiguation, as established in Section 4.4. Disambiguation accuracy does not improve with model size; the 7B model is no more reliable than the 1.5B model at distinguishing semantically similar candidate functions.

CD+FT-aligned exceeds CD at all four model sizes. The fine-tuning benefit is non-monotonic: largest at 0.5B (+7.75 pp), decreasing at 1.5B (+3.75 pp) and 3B (+2.00 pp), then recovering at 7B (+4.00 pp). The 0.5B model benefits most, consistent with a format-learning effect that is more impactful when the base capability is lower. The 3B model is near a local ceiling where the xlam training examples add marginal semantic value beyond what CD alone provides. The 7B model has sufficient capacity to extract a stronger training signal. Fine-tuning does not reduce the model-size gap: the 3B–7B gap under CD+FT-aligned is 10.00 pp, slightly wider than the 8.00 pp gap under CD alone.

4.8.1 Cross-Family Generalization

The size sweep above varies model scale within a single family. To test whether the constrained-decoding result depends on Qwen 2.5 specifically, Config CD is evaluated on BFCL v4 `simple_python` for the four contrast checkpoints introduced in Section 3.3.2, using the identical pipeline and changing only the served model. Table 4.14 reports the result against the Qwen 2.5 size ladder.

Across the four contrast families the CD accuracies track parameter count and known instruct quality rather than clustering by lab. Llama 3.2 3B reaches 62.50%, within 2.25 pp of the same-size Qwen 2.5 3B (64.75%), so the constrained-decoding benefit at the 3B scale is not a Qwen artefact. The two near-1B checkpoints, Llama 3.2 1B (60.50%) and Gemma 3 1B (55.50%), both land above Qwen 2.5 0.5B (51.50%) and bracket Qwen 2.5 1.5B (62.25%), and their internal ordering mirrors the general capability gap between the two instruct models. Phi-4-mini (3.8B) is the strongest contrast point at 68.25%, above the same-range Qwen 2.5 3B and within 4.50 pp of Qwen 2.5 7B, consistent with its reputation for performing above its parameter count.

The result supports the family-robustness claim that motivates the contrast checkpoints

Table 4.14: BFCL v4 `simple_python` AST accuracy for Config CD across model families (400 test cases, guided decoding, bfloat16, out-of-the-box instruct checkpoints). The Qwen 2.5 size ladder is shown for reference. Gemma 3 4B is excluded: it loads as a multimodal model on the frozen stack (vLLM 0.8.5), under which `guided_json` is not enforced, so the constrained-decoding protocol cannot be measured for it.

Model	Approx. size	Correct	Accuracy (↑)
Gemma 3 1B-Instruct	1.0B	222/400	55.50%
Llama 3.2 1B-Instruct	1.2B	242/400	60.50%
Llama 3.2 3B-Instruct	3.2B	250/400	62.50%
Phi-4-mini-Instruct	3.8B	273/400	68.25%
Qwen 2.5 0.5B (CD)	0.5B	206/400	51.50%
Qwen 2.5 1.5B (CD)	1.5B	249/400	62.25%
Qwen 2.5 3B (CD)	3.0B	259/400	64.75%
Qwen 2.5 7B (CD)	7.0B	291/400	72.75%

(Section 3.3.2): constrained decoding lifts sub-4B instruct models from the near-zero baseline floor (Section 4.8) into the 55–68% band on this category across three additional training pipelines, with accuracy ordered by model capacity rather than by training lineage. The evaluation is confined to Config CD on `simple_python`; the quantized, fine-tuned, and agentic configurations remain on Qwen 2.5 (Section 5.3.1).

Fig. 4.6 summarises both results on a single accuracy-versus-size axis. Accuracy rises monotonically with scale for every technique, but the constraint mechanism (CD, CD+schema) carries the accuracy at all four sizes while the raw base (B) stays on the near-zero floor; CD+Q tracks CD so closely that the quantization penalty is barely visible. The four cross-family CD points fall within the Qwen 2.5 CD band at the corresponding parameter count, ordered by model capacity rather than by training lineage.

4.9 Extended Function-Calling Categories

This section reports results for the three harder BFCL v4 categories: `multiple`, `parallel`, and `parallel_multiple`. Each category isolates a different aspect of tool-calling complexity beyond single-function, single-call generation.

4.9.1 Multiple-Function Selection

The `multiple` category presents several candidate functions and requires selecting one and generating a single call. Table 4.15 shows accuracy for CD and CD+FT-aligned across model sizes.

Table 4.15: BFCL v4 AST accuracy on the `multiple` category (200 test cases). CD and CD+FT-aligned at all four sizes.

Size	CD	CD+FT-aligned
0.5B	42.0% (84/200)	55.5% (111/200)
1.5B	53.5% (107/200)	61.0% (122/200)
3B	62.0% (124/200)	60.5% (121/200)
7B	70.5% (141/200)	70.5% (141/200)

CD alone scales from 42.0% at 0.5B to 70.5% at 7B. CD+FT-aligned improves over CD by 13.5 pp at 0.5B and 7.5 pp at 1.5B, but the gap closes to within noise at 3B (60.5% vs.

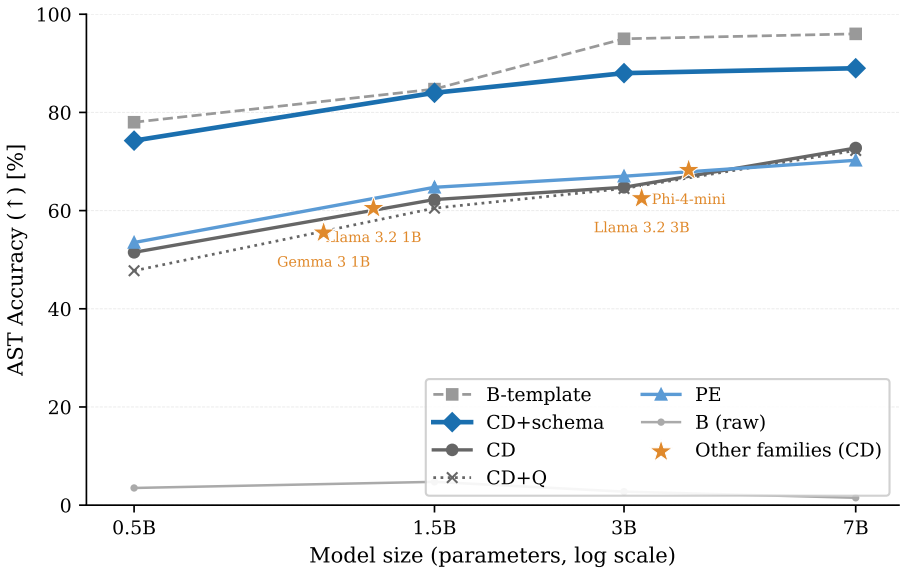

Figure 4.6: BFCL v4 `simple_python` AST accuracy versus Qwen 2.5 model size (log scale) for the main configurations (Table 4.13). Stars mark Config CD on four other instruct families at their approximate parameter count (Table 4.14); they fall within the Qwen 2.5 CD band by capacity, not by lab. CD+Q overlaps CD throughout, showing the negligible INT4 quantization penalty above 1.5B.

62.0%) and vanishes entirely at 7B, where both configs reach 70.5%. The pattern mirrors the `simple_python` sweep: fine-tuning helps most at smaller sizes where the model has limited capacity to infer the required format from the prompt alone; at 7B the base model with CD already saturates the category.

4.9.2 Parallel Function Calling

The `parallel` category requires generating two simultaneous calls to the same function. Under the initial single-extraction design (Chapter 3), accuracy is 0.0% at every model size from 0.5B to 7B. The parallel selection stage emits a set of distinct function names and produces exactly one argument object per selected name, and every `parallel` case provides a single distinct function, so the pipeline emits one call by construction. Inspection of the raw outputs confirms a single call where two are required, with argument content otherwise correct; the failure is in call count, not argument values. This is a structural property of the per-function design, not a soft accuracy limitation that scale or further training would gradually close.

The joint multi-call schema removes this constraint. When the model instead generates a JSON array of complete call objects in one guided pass, the same function may recur with distinct arguments, and parallel accuracy increases with model size rather than collapsing to zero (Table 4.16). Plain CD rises from 9.5% at 0.5B to 79.0% at 7B; CD+FT-aligned reaches 72.0% at 7B. The limitation was therefore in the output schema, not model scale: once the schema admits repeated calls, accuracy on the category rises with size as in the other categories.

4.9.3 Parallel Multi-Function Calling

The `parallel_multiple` category combines both challenges: multiple simultaneous calls drawn from different candidate functions. Under the joint multi-call schema it is evaluated across all four sizes alongside `parallel`; Table 4.16 reports both.

Table 4.16: BFCL v4 AST accuracy on the `parallel` and `parallel_multiple` categories under the joint multi-call schema (200 test cases each, Qwen 2.5).

	`parallel`		`parallel_multiple`	
Size	CD	CD+FT-aligned	CD	CD+FT-aligned
0.5B	9.5% (19/200)	45.5% (91/200)	3.0% (6/200)	16.5% (33/200)
1.5B	64.0% (128/200)	58.0% (116/200)	46.0% (92/200)	33.5% (67/200)
3B	74.5% (149/200)	41.5% (83/200)	63.0% (126/200)	38.0% (76/200)
7B	79.0% (158/200)	72.0% (144/200)	72.5% (145/200)	60.0% (120/200)

Plain CD on `parallel_multiple` rises from 3.0% at 0.5B to 72.5% at 7B. At every size, CD+FT-aligned scores below plain CD on this category, and on `parallel` at 3B and 7B as well: these are the only categories where fine-tuning underperforms plain CD. Both configurations produce multi-call responses at comparable rates, so output structure is not the bottleneck. The gap comes from semantic errors in the CD+FT-aligned outputs: argument values copied from the first call to the second, notation normalised to match the training corpus (e.g. `x**2` substituted for `x^2`), and the occasional dropped call in three-call responses. These are biases from the xlam training distribution that reduce argument fidelity on tasks requiring several simultaneous calls. The CD+FT-aligned numbers are also non-monotonic in size (3B falls below 1.5B on both categories), consistent with the aligned-adapter degradation noted in the size sweep (Section 4.8).

At 7B under plain CD, `parallel` (79.0%) now exceeds `parallel_multiple` (72.5%), reversing the ordering seen under the single-extraction design, where `parallel` was 0.0% and `parallel_multiple` non-zero because distinct-function prompts let the old pipeline emit more than one call. Under the joint schema both categories can emit repeated calls, so the residual gap reflects only that `parallel_multiple` additionally requires selecting among several candidate functions, not a structural barrier specific to same-function repetition.

4.9.4 Capability Decomposition

The three categories define a ladder for the techniques evaluated in this thesis. Function selection under CD+FT-aligned is largely solved at 7B (70.5% on `multiple`). Same-function parallel calling is limited by the output schema rather than by model capability: under the single-extraction design it is 0.0% at every size, while under the joint multi-call schema it ranges from 9.5% at 0.5B to 79.0% at 7B. The combined task (`parallel_multiple`) tracks just below `parallel` (72.5% at 7B, 3.0% at 0.5B under plain CD), as it adds candidate-function selection on top of multi-call emission. Across both categories the residual gap is dominated by call-count and argument errors that narrow with scale; format-aligned fine-tuning does not help and slightly lowers argument fidelity, so closing the gap would require training data targeting simultaneous multi-call generation, not format alignment alone.

5 Discussion

This chapter interprets the experimental results in terms of the thesis questions. The central distinction is between structural failures, which can be removed by constrained decoding, and semantic value errors, which require better schema information, training data, or routing. The chapter first analyses this distinction, then discusses cost-benefit trade-offs, limitations, practical deployment implications, and the relation to prior work.

5.1 Impact of Constrained Decoding vs. Model Scale

The gap between Base and CD is almost entirely a format compliance gap. Under the strict whole-completion parser this gap is 71.25 percentage points; under the lenient first-object parser used in the isolation arm, the 7B same-parser accuracy gain is 10.75 pp. Of the 394 strict-parse failures in Base, virtually all are structural: the model produces conversational text, multiple JSON objects, or empty output rather than a single valid function call. The six correct outputs in Base are semantically accurate but happen to be formatted correctly by chance. The model is not unable to reason about the queries; it cannot express its answers in the required structured format without guidance.

This distinction has a direct consequence for interpreting the frontier comparison. CD achieves 72.75% on BFCL v4 `simple_python`, 6.00 pp below the weakest frontier entry on the same category (GPT-5-mini, 78.75%) and 18.25–25.00 pp below flagship frontier models (91.00%–97.75%) [32].[1] Constrained decoding enforces valid JSON and valid function names by construction, which is precisely what frontier models' native tool-calling APIs also do. Once format compliance is equalized, a 7B model and a frontier model are distinguished only by argument value accuracy. There the constrained pipeline trails the flagship models by 14 pp or more, while the same model under its native template (B-template, 96.00%) is competitive with them: the deficit belongs to the model-agnostic pipeline, not to the model.

The comparison is specific to the `simple_python` category. Each test case provides exactly one candidate function, eliminating function selection as a source of error. With one candidate and `guided_choice`, selection accuracy is 100% regardless of model size. The remaining 27.25% failure rate is entirely in argument values: wrong optional parameter defaults, numeric precision errors, and string format mismatches. Frontier models make the same class of errors at a lower rate; the gap of 25.00 percentage points to the best frontier model (Claude Sonnet 4.5, 97.75%) reflects a higher frequency of correct value conventions under prompts that carry the full schema detail, not a qualitatively different failure mode.

The implication for RQ1 is that the raw gap of approximately 96 percentage points between the model-agnostic baseline (7B Base at 1.5%) and the best frontier model (Claude Sonnet 4.5 at 97.75%) collapses to approximately 25 percentage points with constrained decoding alone, on this benchmark category. Two readings of this gap must be kept apart. The 1.5% baseline characterises a generic integration without the model's native tool-calling template; the B-template control measures the same model at 96.00% with native prompting and free generation (Chapter 3), so the 96 pp figure measures the cost of naive integration, not a capability deficit of the model. What the constrained decoding result adds is that

[1] Frontier model scores are the Python Simple AST sub-category of the BFCL v4 leaderboard, accessed 2026-06-04 and re-verified 2026-06-10; see the comparability note in Section 4.5.

format compliance can be restored structurally, with guarantees independent of any model-specific template. Scale is not the bottleneck for format compliance. The residual gap is a semantic problem, but the B-template control constrains its interpretation: given its native template and the full schemas, the model resolves most of these cases. Config CD+schema isolates the information-loss component directly: enriching the constrained prompt with parameter descriptions, enumeration values, and defaults raises accuracy from 72.75% to 89.00% (+16.25 pp, McNemar $p < 0.00001$; Section 4.2.3), showing that the majority of the residual gap after plain CD is consistent with missing schema detail rather than only the model's learned value priors. The remaining 7.00 pp gap between CD+schema (89.00%) and B-template (96.00%) bounds the effect of factors not isolated by CD+schema. Because B-template also changes the prompt format, tool-call delimiters, and parser target, the gap should be read as a residual integration difference rather than as a single identified cause.

5.1.1 Effect of Model Scale on CD Performance

The format compliance failure that accounts for 98.5% of Base errors is not resolved by scale. Without constrained decoding, accuracy across the 0.5B, 1.5B, 3B, and 7B Qwen 2.5 models ranges from 1.50% to 4.75%, despite a 14-fold difference in parameter count. The six correct outputs in 7B Base and the fourteen correct outputs in 0.5B Base are produced by chance; the underlying failure mode, malformed or conversational output rather than a single valid JSON object, is present at every scale.

With constrained decoding applied, the accuracy curve follows a consistent scaling trend: 51.50% at 0.5B, 62.25% at 1.5B, 64.75% at 3B, and 72.75% at 7B. The steepest gain occurs between 0.5B and 1.5B (+10.75 pp); the 1.5B-to-3B step is flatter (+2.5 pp); the 3B-to-7B step recovers to +8.0 pp. Constrained decoding shifts the entire curve upward without altering its shape, consistent with the view that format compliance and semantic accuracy are separable and that the scaling relationship governs the latter.

The AWQ quantization penalty is inversely related to model size. The accuracy loss relative to the bfloat16 baseline is 3.75 pp at 0.5B, 1.75 pp at 1.5B, 0.25 pp at 3B, and 0.50 pp at 7B. At 3B and above, the INT4 approximation falls within run-to-run variance. At 0.5B, the 3.75 pp loss reflects the reduced parameter redundancy available to absorb quantization error, consistent with the sensitivity of AWQ calibration on small weight matrices [18]. Quantization is therefore safe for deployment above 1.5B and introduces a meaningful but bounded cost at 0.5B.

Few-shot prompting (Config PE) reverses direction at 7B. At 0.5B, 1.5B, and 3B, the three in-context examples improve accuracy by 2.0 to 2.5 pp over the CD baseline, suggesting the examples compensate for weaker internal argument-value representations at smaller scales. At 7B, the same examples reduce accuracy by 2.5 pp. The approximately 300 additional context tokens introduced by the examples appear to compete with the test case for attention at 7B, an effect consistent with the prompt-length sensitivity reported for large-scale in-context learners [3]. The practical implication is that the best no-training configuration is size-dependent: Config CD alone at 7B, and Config PE at sizes up to 3B.

5.2 Cost-Benefit Analysis of Each Method

Constrained decoding yields the highest structural return of any technique evaluated. It guarantees parseable outputs with no training, no extra parameters, and no additional generated tokens. The accuracy gain is 71.25 percentage points relative to the strict generic floor and 10.75 pp relative to the lenient 7B parser baseline. The per-step cost of guided decoding is not zero: schema compilation and logit masking add overhead that grows with schema complexity, although the finite-state machine approach keeps the per-token

cost low when schemas are compiled ahead of time [31]. Measured on 100 sequential `simple_python` cases (Section 4.2.6), constrained decoding (0.878 s mean per call) is 8× faster than the unconstrained baseline (6.995 s mean): the unconstrained model appends prose, markdown, and additional JSON objects, generating far more tokens; the constraint terminates generation at the closing brace. Constrained decoding is therefore not a latency tax but a latency gain at 7B. The structural gain comes from logit masking at each decoding step, which eliminates the malformed output that accounts for 98.5% of failures in Base [31, 16].

Schema-enriched constrained decoding extends the plain CD result by supplying the parameter descriptions, type constraints, and enumerated values that the standard pipeline omits from its prompts. Config CD+schema achieves 89.00% (+16.25 pp over plain CD, McNemar $p < 0.00001$; Section 4.2.3), the highest accuracy of any configuration in the evaluation and the only constrained result to fall within the lower BFCL v4 Python Simple AST leaderboard band without modifying model weights. No training, no additional generated tokens, and no latency overhead are introduced: the measured per-call latency for CD+schema (0.704 s mean) falls within the same range as plain CD and does not exceed the unconstrained baseline. The implementation cost is maintaining enriched prompt templates that include parameter descriptions, type annotations, and enumerated defaults for each tool. The cost-benefit ratio is the most favorable in the evaluation, and the gain scales with schema richness: tools with detailed descriptions and enumerated defaults expose more of the missing vocabulary that schema enrichment recovers.

AWQ INT4 quantization adds a further infrastructure benefit at near-zero accuracy cost [18]. The −0.5 percentage point change (two test cases) falls within run-to-run variance. Memory is reduced by 63.5% (14.25 GiB to 5.20 GiB) and model loading time by 83.5% (212 s to 35 s). The cost-benefit ratio is strongly positive: substantial deployment benefits at negligible accuracy cost.

Few-shot prompting (PE) and chain-of-thought reasoning (CD+Q+ITC) both produce negative outcomes [3, 30]. PE reduces accuracy by 2.5 percentage points with a context overhead of approximately 300 tokens per query. CD+Q+ITC reduces accuracy by 6.75 percentage points while increasing per-case inference time proportionally to the additional tokens generated by the reasoning trace (Chapter 4 details the per-case flip analysis and the three mechanisms behind the CoT regression). Both techniques share the same root cause: the residual 27% failure rate after constrained decoding is a mismatch between the values the pipeline elicits and the benchmark's ground truth conventions. The B-template control indicates that a substantial part of this residual stems from schema information missing from the pipeline's prompts: neither three in-context examples nor an explicit reasoning step substitutes for the absent parameter descriptions, and the reasoning step actively moves values away from the benchmark's conventions. Two independent negative results with the same directional outcome establish that prompt-level additions within the pipeline cannot address this failure class; supplying the missing schema detail itself, as the native template does, is a different and evidently more effective intervention.

This finding aligns with the scope conditions reported by Wei et al. [30]: chain-of-thought prompting improves performance on tasks requiring multi-step arithmetic or symbolic decomposition, not on tasks where the bottleneck is value recall. Argument extraction is not a reasoning problem in that sense; an explicit reasoning step directs compute at the wrong subproblem [26].

Retrieval-augmented generation produces the largest accuracy drop: −24.5 percentage points relative to CD+Q, despite a retrieval recall@5 of 97.2% [17]. The bottleneck is

not retrieval quality but candidate disambiguation. When multiple semantically similar functions are retrieved, the 7B model cannot reliably identify the one matching the user's intent. The infrastructure cost of RAG is substantial (an offline FAISS index and an embedder at query time), making its negative accuracy outcome particularly unfavorable.

The disambiguation failure identified here is avoided by an alternative design in Gorilla [22], which incorporates retrieval at training time rather than inference time. By generating training examples from retrieved API documentation, Gorilla teaches the model to use retrieved content as a grounding signal without exposing it to multiple similar candidates at query time. This sidesteps the inference-time disambiguation problem entirely, at the cost of coupling the model to a fixed API catalog. The configuration evaluated in this thesis uses semantic similarity retrieval at inference time with no training signal on candidate disambiguation, which may explain the low selection accuracy despite high retrieval recall@5 (97.2%). The 66% of all test cases whose output is identical to the non-RAG prediction (Fig. 4.3) confirms that retrieval adds overhead without changing the model's output for the majority of cases. A design that combined inference-time retrieval with a fine-tuning signal on tool selection from retrieved candidates would be a natural next step.

LoRA fine-tuning on the `xlam-function-calling-60k` dataset [35, 12] produces a regression of 3 percentage points under CD+FT relative to CD. The regression is attributed to a format mismatch between the training distribution, which encodes assistant turns as Python call syntax, and the evaluation pipeline, which expects a JSON argument object. The technique is not ruled out by this result. The format mismatch hypothesis is confirmed by CD+FT-aligned, which retrains on the same xlam content reformatted to match the inference pipeline's JSON output convention exactly. CD+FT-aligned achieves 76.75% (307/400), a gain of 7.00 percentage points over CD+FT v1 and 4.00 percentage points above the CD baseline, making it the best training-based result in the evaluation. Under the corrected lenient parser, plain fine-tuning does not raise unguided format compliance above Base: FT-only reaches 53.00%, below the no-guided Base (62.00%), and FT-aligned-ng is lower still at 41.00%. The 35.75 percentage point gap between FT-aligned-ng (41.00%) and CD+FT-aligned (76.75%) shows that constrained decoding remains the major contributor even after fine-tuning. The same dependence holds across families and sharpens away from the 7B setting: without guided decoding the format-aligned adapters fall to 0.00%–3.75% (Phi-4-mini reaches 0.00%), well below the 39.00%–67.50% retained by the matched instruct checkpoints, so format alignment specialises the model to the constrained-decoding protocol rather than producing a stronger standalone generator.

Taken together, the results support a clear ordering by cost-benefit ratio: schema-enriched constrained decoding (CD+schema, 89.00%) delivers the highest accuracy at zero training cost; plain constrained decoding provides the core structural gain with no overhead; quantization adds substantial memory and loading benefits at near-zero accuracy cost; and format-aligned fine-tuning offers further marginal gains at the highest implementation cost. Prompting and retrieval interventions produce negative outcomes on this failure class.

5.3 Limitations

The results are based on the `simple_python` category of BFCL v4, which is the least demanding of the four BFCL categories [32]. Each test case consists of a single user turn, a single candidate function, and a single expected call. The constrained decoding advantage depends on oracle function provision: `guided_choice` guarantees correct function selection only when the correct function is the sole candidate. Generalisation to harder categories is partial: CD+FT-aligned reaches 70.5% on `multiple` at 7B, and the `parallel` category

exposes a limitation in the output schema rather than in model capability. Under the initial single-extraction design the `parallel` score is 0.0% across all configurations and sizes, and this is not a soft accuracy limitation that more training or a larger model would gradually close; it follows from the structure of the parallel selection stage described in Chapter 3. That stage emits a set of distinct function names and runs one deterministic argument extraction per selected name, so the pipeline can produce at most one call per distinct function. In the `parallel` category, where every required call targets the same single candidate function, the pipeline emits exactly one call by construction; in `parallel_multiple`, where the required calls target distinct functions, the same pipeline produces up to one call per function, which is why multi-call responses are observed there (Section 4.9). Supporting repeated calls to the same function requires an output schema in which a JSON array holds complete call objects with their own argument bindings, or a multi-step execution loop where each iteration emits one call.

The first of these remedies was implemented and evaluated. Replacing the single-extraction stage with a joint multi-call schema, in which the model generates an array of complete call objects in one guided pass, lifts `parallel` from a structural 0.0% to between 9.5% at 0.5B and 79.0% at 7B (Section 4.9). This confirms that the constraint was the schema rather than model scale: once repeated calls are permitted, accuracy on the category depends on size in the same way as the other categories. The result therefore identifies a boundary of the single-extraction architecture and the effect of moving past it, rather than a permanent ceiling on the task; same-function parallel calling at the smallest sizes remains hard, so the category is not solved outright.

The size sweep and most ablation configurations use a single model family (Qwen 2.5) across four sizes. Constrained decoding is the exception: it is also evaluated on four other families (Section 4.8.1), where its structural benefit holds, so that finding has external support. The remaining directional findings, such as the ordering of techniques by accuracy, the size-dependence of the PE effect, and the quantization penalty profile, are observed consistently within Qwen 2.5 only. Whether they transfer to other SLM families with different training regimes (e.g. Llama 3.2, Phi-4) is not established and should not be assumed.

The cascade architecture is the motivating deployment context for this evaluation, but no direct cascade experiment is reported. The benefit fractions ($p \approx 0.72$–0.77) cited for each configuration are extrapolated from single-model BFCL accuracy; actual cascade economics depend on routing accuracy, latency characteristics, and the query distribution of the production workload, none of which were measured.

The BFCL ground truth encodes specific value conventions that are often arbitrary from a deployment perspective. A model that produces `"gasoline"` where the ground truth expects `"gas"` is scored as incorrect despite both being valid in a real API call. The 27% residual failure rate after constrained decoding may overestimate the practical error rate for production use, where value conventions are defined by the API schema rather than by a benchmarking dataset.

Multi-step performance (τ-bench: 4.35% pass rate) is evaluated for CD at 7B and for CD and CD+FT-aligned across the size sweep, but not for the full configuration set. The τ-bench result establishes that single-call accuracy does not predict multi-step agentic performance; the remaining no-training configurations were not evaluated on τ-bench because the bottleneck identified is multi-turn planning rather than argument formatting.

5.3.1 Threats to Validity

Several factors limit the generalisability of the findings.

Baseline construction and schema information loss. The Base configuration is a minimal model-agnostic integration that does not use Qwen 2.5's native tool-calling prompt template. The B-template control measures the same model at 96.00% (384/400) under native prompting with free generation (Table 4.1), so the headline gap to frontier models in RQ1 quantifies the cost of naive integration rather than an intrinsic capability deficit. The information-loss component of this gap is now quantified directly by Config CD+schema (Section 4.2.3): enriching the constrained prompt with the full schema detail raises accuracy from 72.75% to 89.00% (+16.25 pp, McNemar $p < 0.00001$), showing that most of the residual after plain CD is consistent with absent descriptions, enumerations, and defaults rather than only with the model's learned value priors. The remaining 7.00 pp gap between CD+schema and B-template is a residual integration gap: B-template also changes prompt format, tool-call delimiters, and parser target, so this control bounds the gap but does not isolate a single cause.

τ-bench user simulator. The user simulator is the agent model itself (Qwen 2.5 7B), whereas the reference τ-bench setup uses a frontier-model simulator. Simulator errors (misstated or omitted task details) deflate pass rates independently of agent capability, so the absolute pass rate conflates agent and simulator failures, and comparisons to published pass rates under stronger simulators are not like-for-like. Three independent runs at identical agent settings (temperature 0.0, seed 42) produced pass rates of 3.48%, 4.35%, and 5.22% (4–6 tasks out of 115). Because the agent is fully deterministic across runs, the 1.74 pp spread is attributable entirely to stochastic simulator turns, bounding the observed simulator-noise contribution at ± 0.87 pp (one task at n=115). The qualitative conclusion, that single-call BFCL gains do not transfer to multi-step agentic completion, is robust to this effect: even at the upper bound (5.22%) the gap to the 72.75% BFCL CD result exceeds 67 pp, and the conclusion holds across all evaluated model sizes (Table 4.10).

Single model family. The size sweep and the quantized, fine-tuned, and agentic configurations use the Qwen 2.5 Instruct family across four sizes. Constrained decoding is additionally evaluated on four other families (Section 4.8.1), where its benefit is family-robust, and the GitHub MCP and format-aligned analyses also span families (Sections 4.7 and 4.4.1). The remaining findings (the ordering of techniques by accuracy, the size-dependence of the few-shot effect, and the quantization penalty profile) are observed only within Qwen 2.5 and may not transfer to other SLM families with different pre-training regimes or instruction-tuning data (e.g. Llama 3.2, Phi-4).

BFCL `simple_python` reliance. The primary evaluation uses the least demanding BFCL category: single-turn, single-candidate queries with no function-selection ambiguity. Results on harder categories are partial (Section 4.9), and the constrained decoding advantage depends on oracle function provision. Generalization to multi-turn or high-ambiguity settings cannot be assumed.

Frontier scores not re-evaluated locally. Frontier model scores are taken from the official BFCL leaderboard rather than re-run under the same evaluation harness. Minor methodology differences (prompt formatting, API tool-call handling) may affect comparability, so the frontier comparisons indicate parity rather than a definitive ranking.

Accuracy as routing proxy. BFCL accuracy is used to proxy the cascade traffic fraction p. After constrained decoding removes structural failures, every SLM output is a valid JSON object naming a known function; structural routing signals are no longer available.

The reported p values are upper-bound estimates that assume an ideal semantic router.

Single-run protocol and run-to-run spread. BFCL results use temperature 0; repeated runs within a fixed software environment produce identical outputs (six-run confirmation for CD+Q), but re-runs spread over the project period varied by one to two cases out of 400, an uncertainty of roughly ±0.5 pp on any single accuracy figure. Differences of this magnitude between configurations (for example, the 0.5 pp CD-to-CD+Q delta) are therefore not individually meaningful, and the paired tests in Chapter 4 are interpreted against this spread. For τ-bench, three runs are completed at 7B but only single runs at smaller sizes, where one-task differences fall within the noise band.

Benchmark convention mismatch. The 27% residual failure rate after constrained decoding may overestimate practical error rates. BFCL ground truth encodes specific value conventions (e.g. `"gas"` not `"gasoline"`, decimal fractions not percentages) that a production API schema would not impose. The residual failures represent convention mismatches, not functional errors, in many cases.

Few-shot example design. The three PE examples (Chapter 3) were authored to target failure classes identified by inspecting the test-split errors, so the design of the few-shot technique is informed by the evaluation set. This biases in PE's favour. The weak and size-dependent PE effect (at best +2.5 pp, and −2.5 pp at 7B) is therefore a conservative estimate rather than an inflated one: the technique fails to help even when its examples are tuned to the observed errors. A held-out development split would be the correct design for a clean, unbiased measurement.

5.4 Practical Implications

The cascade architecture motivating this work assumes that an SLM handles most tool-calling requests locally and escalates only the unreliable cases to a frontier model API. The results make this architecture possible to cost, but only as a projection. No cascade router is implemented here. The estimates below presume a semantic router that can identify likely-wrong SLM outputs; after constrained decoding, structural routing signals are unavailable because every output is valid JSON naming a known function. The reported p values are therefore upper bounds.

As a first-order projection, suppose frontier API calls cost c per query and the local SLM tier has already been deployed, so its marginal cost is treated as small relative to the API call. Under that simplifying assumption, a cascade in which the SLM correctly handles a fraction p of requests saves about $p \cdot c$ per query relative to routing all traffic to the frontier model. CD+Q achieves $p \approx 0.72$, CD+Q+FT-aligned achieves $p \approx 0.74$, and CD+FT-aligned achieves $p \approx 0.77$, all on a single consumer GPU. These are upper-bound traffic fractions, not measured deployment savings, because the router, utilization, batching, energy cost, maintenance cost, and latency service level are not evaluated. The cascade criterion of RQ3 (Chapter 1) adopts $p \geq 0.70$ as its reference point, which all four constrained configurations at 7B exceed under this proxy.

The cost-benefit framing above costs the constrained local tier against a frontier API. For a single-model deployment a stronger local option is available: the same 7B model under its native tool-calling template (B-template) reaches 96.00% on `simple_python` (Table 4.1), above every constrained configuration on the same hardware. For a single frozen Qwen deployment, the native template is therefore the better local baseline. The constrained pipeline is valuable when the goal is model-agnostic structural guarantees across heterogeneous models and schemas: output validity is guaranteed independently of

any model's template, a uniform integration is retained, and the router receives guaranteed-parseable output. The p values reported in this section thus apply to the general multi-model case; for a single frozen model the native template supersedes them.

These values are upper-bound estimates: after constrained decoding is applied, every SLM output is structurally valid, removing structural routing signals. Achieving p close to the measured accuracy requires a semantic routing mechanism that identifies likely semantic failures before or after inference; such a mechanism is not evaluated in this thesis. The marginal benefit of each additional technique is therefore expressible directly in terms of API cost reduction: a 2.0 pp gain from adding quantized fine-tuning on top of CD+Q eliminates 2.0% of remaining frontier API calls per unit time at high query volume.

Whether this gain justifies the training investment depends on query volume. At low throughput, the API cost reduction does not accumulate quickly enough to recover the fine-tuning compute cost; the no-training CD+Q configuration is preferable. At high throughput, the per-query saving compounds and the break-even point moves earlier in the deployment lifecycle. The break-even query volume is given by $N = C_{\text{train}}/(\Delta p \cdot c_{\text{frontier}})$, where C_{train} is the one-time training cost, Δp is the accuracy gain, and c_{frontier} is the frontier API cost per call. For CD+Q+FT-aligned versus CD+Q, $\Delta p \approx 0.02$. The two cost terms can be grounded in current list prices rather than left illustrative. A frontier tool call to GPT-4o is billed at \$2.50 per million input tokens and \$10.00 per million output tokens.[2] A representative `simple_python`-style call consisting of roughly 2,000 input tokens (system instruction, tool schemas, and user query) and 150 output tokens (a single JSON call) therefore costs $c_{\text{frontier}} \approx \$0.0065$. The one-time training cost is the approximately 4 GPU-hours of the LoRA run priced at a current A100 80 GB cloud rate of about \$1.50 per GPU-hour,[3] giving $C_{\text{train}} \approx \$6$. These figures place the break-even point at $N = C_{\text{train}}/(\Delta p \cdot c_{\text{frontier}}) \approx 46{,}000$ queries. The number is not a deployment cost estimate: it omits local serving utilization, queueing, energy, monitoring, router cost, and engineering maintenance. It is also highly sensitive to the escalation target. If the frontier tier is a cheaper model such as GPT-4o-mini (\$0.15 and \$0.60 per million input and output tokens), the same call costs only $c_{\text{frontier}} \approx \$0.0004$ and the break-even rises above 750,000 queries; the trained tier pays for itself only when the model it displaces is an expensive one. At lower frontier costs or smaller query volumes the no-training configuration remains preferable; at higher volumes against a frontier-priced target the investment recovers quickly. The 2.0–5.0 pp accuracy range separating the local-tier configurations represents a concrete trade-off space.

The residual failure classes common to all configurations (wrong optional parameter defaults, numeric precision mismatches, and string format conventions) define the queries that will be escalated regardless of which configuration is deployed. Structural output validation, which would be the natural first routing signal, is ineffective after constrained decoding is applied: every output is valid JSON naming a known function. The relevant routing signals are semantic: query-level classification of likely hard cases before inference, or token-probability confidence scoring after inference. Both are tractable extensions of the evaluated pipeline and represent the primary mechanisms for narrowing the escalation boundary beyond what the optimization techniques evaluated here can achieve.

Concretely, such a router would sit between the constrained pipeline and the frontier API and operate on two signals. The first is a pre-inference query classifier: a lightweight model,

[2] https://openai.com/api/pricing/, accessed 2026-06-28.

[3] Representative on-demand rate for a single A100 80 GB; community-cloud providers quote \$1.19–\$1.49 per GPU-hour (https://www.runpod.io/pricing, accessed 2026-06-28).

for example a sentence embedding of the user query and the candidate tool schema followed by a logistic regression, trained on the per-case correct/incorrect labels already collected in this evaluation to predict the probability that the SLM will produce a semantically correct call, and escalating queries above a risk threshold without running the SLM at all. The second is a post-inference confidence signal derived from the decoder itself. Although constrained decoding guarantees that every output is valid JSON, the token-level log-probabilities under the guided mask are not uniform: the mean log-probability over the extracted argument tokens, or the entropy at the masked argument positions, provides a calibratable score that a threshold can convert into a route decision. A complementary post-inference signal is self-consistency, in which the argument extraction is sampled more than once and disagreement among the samples triggers escalation. Because the residual failure classes are systematic (optional-parameter defaults, numeric precision, and string-format conventions) rather than random, a classifier trained on these labelled cases is the more promising of the two: it targets a structured rather than a diffuse error distribution. The threshold in either case sets the operating point on a precision-recall trade-off between API cost saved and incorrect calls served locally, and its calibration is the design parameter that determines how close the realised p comes to the upper bounds reported above.

Fine-tuning alignment is the binding constraint on the achievable accuracy ceiling. The xlam experiment demonstrates that a general-purpose function-calling corpus with a different output convention actively degrades performance under constrained decoding: the model learns value priors from the training distribution that conflict with the evaluation pipeline's format. This means that the fine-tuning gain is not portable across pipelines. A production fine-tuning run must generate training examples from the target deployment's own tool schemas; otherwise the format mismatch replaces the original semantic failure class with a new one.

5.5 Relation to Prior Work

The results reported in this thesis intersect four bodies of prior work: constrained generation, training-based tool adaptation, retrieval-augmented generation for tool selection, and multi-step agentic evaluation.

Constrained generation. Willard and Louf formalised constrained generation using finite-state machines compiled from JSON schemas, establishing that format compliance can be guaranteed at each decoding step with low per-step overhead [31]. The Base-to-CD comparison confirms this guarantee in practice: every failure involving malformed or conversational output under the strict Base parse is eliminated, and the residual 27.25% failure rate is attributable entirely to argument value errors. This validates the scope claim in Willard and Louf: constrained decoding solves the format compliance problem completely but cannot address semantic accuracy. The strict-parser accuracy recovery is 71.25 pp at 7B, but the same-parser gain over the lenient first-object baseline is 10.75 pp. The value of constrained decoding here is therefore the structural guarantee first and the accuracy margin second; models or parsers that already recover valid JSON would show a smaller measured accuracy benefit. The negative chain-of-thought interaction observed in Section 4.2 also connects to the study of Tam et al. [27], which reports that format restrictions can degrade reasoning performance: the present results show the complementary direction, namely that adding explicit reasoning under a format constraint degrades value accuracy when the bottleneck is convention recall rather than reasoning. Newer structured generation engines such as XGrammar [6] reduce the per-step overhead of grammar-constrained decoding but do not change the accuracy trade-offs analysed here.

Training-based tool adaptation. Gorilla demonstrated that fine-tuning a LLaMA model on retrieved API documentation can outperform GPT-4 on API-calling tasks, establishing that training data relevance can compensate for parameter count disadvantage [22]. The key mechanism is alignment between the model's training context and its inference context. This thesis extends that insight from API source selection to output format alignment. Two LoRA adapters trained on the same xLAM dataset [35] with identical hyperparameters differ by 7.0 pp on BFCL solely because their training output formats conflict or match the inference pipeline's JSON target. The format alignment effect is of comparable magnitude to the improvements Gorilla reports from retrieval-augmented training, suggesting that output format compatibility is as important a dimension of training data design as content relevance.

A broader survey of small language models for agentic systems identifies Qwen 2.5 7B as one of the strongest open-weight SLMs for tool use and notes that systematic combinations of constrained decoding, quantization, and fine-tuning remain underexplored [25]. The cumulative evaluation design adopted in this thesis directly addresses that gap, providing comparative measurements across five technique combinations applied to a single model family over four model sizes.

Retrieval-augmented generation for tool selection. Lewis et al. showed that retrieval-augmented generation improves performance on knowledge-intensive tasks by supplying relevant context at inference time [17]. The RAG evaluation here reveals a limitation of this mechanism for tool selection: retrieval recall@5 of 97.2% produces a -24.5 pp accuracy drop relative to CD+Q rather than an accuracy gain, because the bottleneck shifts from retrieval quality to candidate disambiguation. Retrieved candidates are semantically similar by construction, and the model must distinguish between them on fine-grained semantic grounds that single-call training does not develop. This failure mode is consistent with ToolLLM's observation that model performance degrades as the number of candidate APIs grows [23]. Gorilla's training-time retrieval approach avoids the problem by teaching the model to handle similar candidates during training; inference-time retrieval without a corresponding disambiguation signal does not produce the same effect.

Multi-step agentic evaluation. The 68.4 pp gap between BFCL single-call accuracy (72.75%) and τ-bench pass rate (4.35%) reveals a discontinuity between structured output generation and multi-step reasoning. The ReAct framework and related agentic approaches report a similar pattern: models that produce correct individual tool calls frequently fail to maintain coherent task state across multi-turn interactions [34]. The τ-bench size sweep confirms that this gap is not closed within the 0.5B–7B range: pass rates span 1.74% to 4.35% across all sizes and both configurations tested, with no consistent scale dependence. The failure mode in the agentic setting is not format compliance, which constrained decoding ensures, but multi-turn planning: tracking which database fields have been retrieved, interpreting tool return values, and selecting the correct next action across five to fifteen steps. These capabilities scale with model size more slowly than single-call argument extraction and are not addressed by any of the post-training or inference-time techniques evaluated in this thesis.

6 Conclusion

6.1 Answering the Research Questions

This chapter returns to the three research questions and summarizes what the experiments show. The central finding is that constrained decoding converts tool calling from a format-compliance problem, which is fixable structurally at inference time, into a problem of semantic argument-value error. That residual error can be reduced by restoring schema detail or, more modestly, by format-aligned training. The answers below quantify these effects and then translate them into deployment guidelines and future work.

RQ1. *What is the performance gap between unoptimized small language models (0.5B–7B parameters) and frontier models on tool-calling benchmarks?*

Across all four Qwen 2.5 Instruct sizes evaluated on BFCL v4 `simple_python`, the model-agnostic baseline achieves 1.5–4.75% under the strict whole-completion parser, with no consistent trend across model scale. The dominant failure mode is format non-compliance: 394 of 400 7B base outputs fail the strict parse rather than producing a semantically wrong structured call, and the same pattern holds at all sizes. This gap quantifies the cost of a naive generic integration and strict parse contract rather than an intrinsic capability deficit: the same 7B run reaches 62.00% when the first complete JSON object is accepted, and the B-template control, which prompts the same model through its native tool-calling template with free generation and full schemas, reaches 96.00% (384/400) on the same test cases. Frontier models range from 78.75% (GPT-5-mini) to 97.75% (Claude Sonnet 4.5) on the same category, with flagship models at 91.00% or above [32]; the B-template control falls inside that flagship band. Constrained decoding closes the strict-parser integration gap at every size without modifying model weights, raising accuracy to 51.50% (0.5B), 62.25% (1.5B), 64.75% (3B), and 72.75% (7B), while guaranteeing that the outputs are structurally parseable. At 7B, the same-parser gain over the lenient unguided baseline is 10.75 pp. At 7B, the residual gap to the best frontier model is approximately 25 pp, and 6.00 pp to the weakest frontier entry (GPT-5-mini, 78.75%). The residual gap is in argument value accuracy, where frontier models and the native-template configuration share an advantage attributable to prompts that carry the full schema detail and to stronger learned associations for specific numeric, string, and format conventions.

RQ2. *What is the marginal contribution of each optimization technique (constrained decoding, schema enrichment, prompt engineering, chain-of-thought prompting, retrieval-augmented generation, quantization, and LoRA fine-tuning) when applied cumulatively to small models for tool calling?*

Constrained decoding is the dominant optimization technique because it gives a model-agnostic structural guarantee: every output conforms to the required schema, without training and without generating additional tokens. Relative to the strict whole-completion parser floor, this corresponds to a +48–+71 pp recovery across the 0.5B to 7B size range. At 7B, where the same unguided run is also scored with the lenient first-object parser, the same-parser accuracy gain is +10.75 pp (62.00% to 72.75%).

AWQ INT4 quantization reduces model memory by 63.5% (14.25 GiB to 5.20 GiB at 7B) and startup time by 83.5% (212 s to 35 s) at a cost of at most 0.5 pp above 1.5B, within run-to-run variance; at 0.5B the penalty is 3.75 pp.

Few-shot prompting (PE) produces a size-dependent effect: accuracy improves by 2.0–2.5 pp at 0.5B–3B but decreases by 2.5 pp at 7B, indicating that the direction of effect is not consistent across model scale. Chain-of-thought reasoning (CD+Q+ITC) reduces accuracy at every model size by approximately 6 pp at increased per-case inference time from the additional reasoning tokens. Retrieval-augmented generation (CD+Q+RAG) reduces accuracy at every model size, with decreases of 16–26 pp relative to CD+Q, despite a retrieval recall@5 of 97.2%; the loss is attributable to function disambiguation failure rather than retrieval quality. The disambiguation failure does not improve with model scale: the 7B model is no more reliable than the 1.5B model at distinguishing semantically similar candidate functions.

LoRA fine-tuning on `xlam-function-calling-60k` [35] produced a 3 pp regression under CD+FT at 7B, attributed to a format mismatch between the training distribution and the evaluation pipeline. The format mismatch hypothesis is confirmed by CD+FT-aligned, which reformats training outputs to match the inference pipeline exactly. CD+FT-aligned improves accuracy at every size: +7.75 pp at 0.5B, +3.75 pp at 1.5B, +2.00 pp at 3B, and +4.00 pp at 7B (76.75%). CD+Q+FT-aligned applies AWQ INT4 quantization to the format-aligned LoRA-merged model and achieves 74.25% (297/400) at 7B on the same 5.20 GiB footprint. The quantization penalty relative to CD+FT-aligned is 2.5 pp at 7B, larger than the 0.5 pp penalty observed when quantizing the base model, suggesting that AWQ interacts more destructively with fine-tuned weight deltas when the calibration distribution does not match the fine-tuning domain. Paired McNemar tests place the 7B fine-tuning gain at the edge of significance ($p = 0.044$, borderline against the observed run-to-run spread) and the quantized-stack gain within noise ($p = 0.38$). These p-values are diagnostic rather than family-wise corrected across the full ablation grid, so the consistent positive direction across all four model sizes is the primary evidence for the format-alignment effect.

Config CD+schema, which enriches the Stage 2 prompt with parameter descriptions, enumeration values, and defaults while keeping the FSM structural constraint unchanged, raises 7B accuracy from 72.75% to 89.00% (+16.25 pp, McNemar $p < 0.00001$; Section 4.2.3). This is the strongest no-training gain observed in this evaluation and confirms that the dominant component of the residual after plain CD is schema information loss rather than the model's learned value priors.

The finding for RQ2 is that the residual failure rate after constrained decoding is not addressed by generic prompt-level additions at any model size; format-aligned training reduces it modestly, and schema-enriched prompting reduces it substantially by restoring the parameter context that the model-agnostic prompt discards.

RQ3. *Can an optimized small language model configuration provide an upper-bound local tier for cascade-style single-call tool calling on consumer hardware?*

Under the scope defined in Chapter 1, the answer is yes as an upper-bound local-tier result for single-call, schema-constrained tool calling on consumer hardware. It is not a claim of end-to-end cascade viability: the semantic router the cascade requires is not implemented, and constrained decoding removes the structural validity signal that would otherwise drive routing, so the reported traffic fractions are upper bounds under an ideal semantic router (Section 5.4). Config CD+schema achieves 89.00% at 7B (Section 4.2.3), which falls within the lower BFCL v4 Python Simple AST leaderboard band between GPT-5-mini (78.75%) and GPT-4.1 (91.00%). It is the first constrained configuration in this evaluation to reach that band while retaining structural validity guarantees and requiring no model-weight

updates. This comparison is indicative because frontier scores are taken from the public leaderboard rather than re-run under the local harness (Section 4.5).

The result is deliberately narrow. It does not establish broad production-agent viability. On the multi-candidate and parallel BFCL categories, the same model remains below the frontier band (Section 4.9), and on τ-bench retail the pass rate remains 4.35% for CD, showing that single-call gains do not transfer to multi-turn task completion. The B-template control (96.00%) also shows that, for a single frozen Qwen deployment, the native tool-calling template is the stronger local baseline. The constrained pipeline is justified when model-agnostic structural guarantees are required across heterogeneous models and schemas.

Against the cascade proxy criterion (a single consumer GPU and a local tier handling $p \geq 0.70$ of single-call traffic under an ideal semantic router), five configurations meet the proxy threshold. CD+schema achieves 89.00% at full bfloat16 precision (14.25 GiB), fitting on a consumer NVIDIA RTX 4090 (24 GiB) alongside substantial key-value cache capacity. CD+Q achieves 72.25% on 5.20 GiB with no training required. CD+Q+FT-aligned achieves 74.25% on the same 5.20 GiB footprint, while CD+FT-aligned achieves 76.75% and CD 72.75% at full bfloat16 precision (14.25 GiB). The cascade interpretation presumes that residual semantic failures are handled by escalation to a frontier model; the semantic router needed to do this is not implemented in this thesis.

Table 6.1: Main thesis claims, supporting evidence, and scope.

Claim	Evidence	Scope
Constrained decoding removes the dominant structural failure mode	Config B scores 1.50%; Config CD scores 72.75%, with 100% valid function selection on `simple_python`	BFCL v4 `simple_python`; model-agnostic Qwen 2.5 pipeline
Schema detail explains much of the residual after CD	Config CD+schema raises accuracy from 72.75% to 89.00% (+16.25 pp, McNemar $p < 0.00001$)	7B model on BFCL v4 Python Simple AST
Quantization is a practical deployment step above 1.5B	AWQ INT4 reduces 7B memory from 14.25 GiB to 5.20 GiB with a 0.5 pp accuracy change	Qwen 2.5 AWQ variants; single-call BFCL evaluation
Format-aligned fine-tuning helps, but less than schema enrichment	CD+FT-aligned reaches 76.75% at 7B and improves all four Qwen sizes, while CD+schema reaches 89.00% without training	Format-aligned xlam LoRA; BFCL single-call setting
Single-call gains do not solve multi-turn agency	CD reaches 72.75% on BFCL `simple_python` but 4.35% pass rate on τ-bench retail	τ-bench retail with a 7B user simulator

6.2 Practical Deployment Guidelines

The results suggest a decision procedure for teams deploying SLMs in single-call tool-calling systems. The procedure is ordered by practical return: use the strongest available integration first, restore schema detail before training, and only then pay the cost of fine-tuning. The cascade cost model that quantifies the saving from each step is developed

in Section 5.4.

Step 1: Benchmark the native template and decide whether model-agnostic guarantees are required. When the deployed model ships a tool-calling chat template, that template is the zero-cost reference point and should be benchmarked first; in this evaluation it reaches 96.00% with free generation (B-template). For a single frozen Qwen deployment, this is the stronger local baseline. Constrained decoding should be used when structural validity must be guaranteed independently of the model's native template, or when the deployment must support heterogeneous models and schemas under one integration.

Step 2: Apply constrained decoding with full schema detail. The highest-return no-training configuration is CD+schema. Carrying parameter descriptions, enumeration values, and default values into the argument extraction prompt raises 7B accuracy from 72.75% to 89.00% (+16.25 pp, $p < 0.00001$) while preserving the structural guarantee of guided decoding. This should be the default constrained setup whenever tool schemas contain rich metadata. Plain CD remains useful when schemas are sparse or when only type constraints are available.

Step 3: Quantize when memory or cold-start latency matter. AWQ INT4 quantization reduces 7B model memory from 14.25 GiB to 5.20 GiB and startup time from 212 s to 35 s, at a cost of 0.5 pp for the base constrained model. This cost is within run-to-run variance at 7B, making quantization the recommended deployment step when hardware capacity or load time is binding.

Step 4: Treat generic prompting additions as low priority. Few-shot prompting and chain-of-thought reasoning both regress at 7B in this evaluation: −2.5 pp for few-shot and −6.75 pp for chain-of-thought, with additional token cost for the reasoning trace. These methods do not address the residual failure mode after constrained decoding, which is semantic convention matching rather than format learning.

Step 5: Fine-tune only with deployment-aligned data. Format-aligned LoRA fine-tuning improves on plain CD, reaching 76.75% at 7B, but it is weaker than schema enrichment and depends on exact alignment between the training output format and the inference protocol. Fine-tuning should be reserved for cases where schema enrichment is insufficient and training examples can be generated from the target tool schemas and argument conventions. Applying the merged adapter to the quantized model yields 74.25% on the same 5.20 GiB footprint.

Deployment boundary. These guidelines apply to single-call, schema-constrained scenarios analogous to BFCL v4 `simple_python`. They do not establish reliable multi-turn agentic execution. The τ-bench result (4.35% pass rate on retail) shows that applications requiring multi-step tool use need additional planning, state-tracking, and recovery mechanisms beyond the techniques evaluated here.

6.3 Future Work

The τ-bench result (4.35% pass rate on retail) establishes that single-call accuracy does not predict multi-step agentic performance. Improving multi-step reliability is the most important open direction. The failure mode on τ-bench is not argument formatting (constrained decoding handles that) but multi-turn planning and error recovery across 5–15 tool calls. Techniques such as process reward models, chain-of-thought with tool-observation grounding, and task-specific fine-tuning on multi-turn trajectories are natural next steps [33]. A controlled ablation isolating the contribution of each technique to τ-bench

pass rate would determine which combination most effectively closes the gap between the 4.35% result observed here and the pass rates reported for frontier models on the same benchmark. The τ-bench airline domain offers a second evaluation environment that would indicate whether the retail result generalises across task structures and tool catalog sizes.

Several extensions would complete the single-call evaluation grid. Config CD+Q+FT-aligned is evaluated at 7B only. Extending it to 0.5B, 1.5B, and 3B would show whether the 2.5 pp quantization penalty on the fine-tuned model increases at smaller scales. The no-guided isolation variants are also reported only at 7B. Running them across the size range would test whether the unguided floor and the ordering of prompt-level techniques remain stable as capacity decreases.

A smaller controlled ablation would also sharpen the central positive result. Config CD+schema injects three kinds of information at once: natural-language parameter descriptions, enumerated value constraints, and default values. The 16.25 pp gain over plain CD is therefore an aggregate effect. Adding these metadata types one at a time, on the same 400-case split and with no new model runs beyond re-prompting, would attribute the gain to its components. It would also indicate which class of schema metadata is worth authoring for a target tool catalogue.

RAG-based tool selection requires a stronger disambiguation signal than top-5 retrieval provides. The retrieval component itself is not the bottleneck: recall@5 is 97.2% in Config CD+Q+RAG. The failure occurs when the model must choose among semantically similar retrieved candidates. Future work should therefore target the selection step directly, for example with a smaller k, a learned re-ranker, or fine-tuning on tool-selection examples from the target catalogue.

The harder BFCL categories point to a second direction. The joint multi-call schema lifts `parallel` from a structural 0.0% under the original single-extraction design to 79.0% at 7B. This confirms that the original limitation was the output schema. The remaining gap on `parallel_multiple`, where format-aligned fine-tuning lowers accuracy relative to plain CD, suggests that future training data should target simultaneous multi-call generation rather than JSON format alignment alone.

Training data alignment remains the binding constraint on LoRA fine-tuning. A production fine-tuning run should generate examples from the target deployment's own tool schemas and argument conventions, rather than relying only on a general-purpose corpus such as xlam. A systematic comparison of synthetic, domain-matched, and human-curated examples would show how much of the residual semantic error can be closed by data quality alone.

Finally, the cross-family evidence in this thesis is a targeted external-validity check rather than a full replication. Config CD and CD+FT-aligned are evaluated across contrast checkpoints, but the quantized and agentic configurations remain concentrated on Qwen 2.5. Extending AWQ INT4, quantized fine-tuned variants, and τ-bench evaluations across the same contrast families would test whether the deployment conclusions hold beyond the primary model family.

Beyond these specific extensions, the ordering of results points to a broader question about how tool-calling agents should be built. The most effective intervention measured here is not a larger model, more training data, or an extra reasoning step. It is the restoration of schema detail that was already present in the tool definitions but had been dropped from the prompt. A no-training, prompt-level change outperformed format-aligned fine-tuning at a fraction of the engineering cost.

This suggests that making tool interfaces machine-legible may return more reliability per unit of work than further training of the model that consumes them. Complete parameter descriptions, explicit enumerations, and stated defaults are useful because every model behind the schema can inherit them. A fine-tuning gain, as the xLAM result shows, is specific to one model and one output convention. The evidence here comes from a single benchmark category and should not be over-generalised. Still, it suggests that the field may have invested more effort in adapting models to tools than in making tools describe themselves well to models.

Bibliography

[1] Marah Abdin et al. "Phi-4 Technical Report". In: *arXiv preprint arXiv:2412.08905* (2024).

[2] Luca Beurer-Kellner, Marc Fischer, and Martin Vechev. "Prompting Is Programming: A Query Language for Large Language Models". In: *Proceedings of the ACM on Programming Languages* 7.PLDI (2023).

[3] Tom Brown et al. "Language Models are Few-Shot Learners". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020.

[4] Haotian Chen et al. "ToLeaP: Rethinking Development of Tool Learning with Large Language Models". In: *arXiv preprint arXiv:2505.11833* (2025).

[5] Tim Dettmers et al. "QLoRA: Efficient Finetuning of Quantized Language Models". In: *Advances in Neural Information Processing Systems*. Vol. 36. 2023.

[6] Yixin Dong et al. "XGrammar: Flexible and Efficient Structured Generation Engine for Large Language Models". In: *arXiv preprint arXiv:2411.15100* (2024).

[7] dottxt-ai. *Outlines: Structured Text Generation*. 2024. URL: https://github.com/dottxt-ai/outlines.

[8] Gemma Team. "Gemma 3 Technical Report". In: *arXiv preprint arXiv:2503.19786* (2025).

[9] Aaron Grattafiori et al. "The Llama 3 Herd of Models". In: *arXiv preprint arXiv:2407.21783* (2024).

[10] Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. "Distilling the Knowledge in a Neural Network". In: *arXiv preprint arXiv:1503.02531* (2015).

[11] Jordan Hoffmann et al. "Training Compute-Optimal Large Language Models". In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022.

[12] Edward J Hu et al. "LoRA: Low-Rank Adaptation of Large Language Models". In: *arXiv preprint arXiv:2106.09685* (2022).

[13] International Committee of Medical Journal Editors. *Recommendations for the Conduct, Reporting, Editing, and Publication of Scholarly Work in Medical Journals*. 2020. URL: https://www.icmje.org/icmje-recommendations.pdf (visited on 06/09/2026).

[14] Jeff Johnson, Matthijs Douze, and Hervé Jégou. "Billion-Scale Similarity Search with GPUs". In: *IEEE Transactions on Big Data* 7.3 (2019), pp. 535–547.

[15] Jared Kaplan et al. "Scaling Laws for Neural Language Models". In: *arXiv preprint arXiv:2001.08361* (2020).

[16] Woosuk Kwon et al. "Efficient Memory Management for Large Language Model Serving with PagedAttention". In: *Proceedings of the ACM SIGOPS 29th Symposium on Operating Systems Principles*. 2023.

[17] Patrick Lewis et al. "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks". In: *Advances in Neural Information Processing Systems*. Vol. 33. 2020.

[18] Ji Lin et al. "AWQ: Activation-aware Weight Quantization for LLM Compression and Acceleration". In: *Proceedings of Machine Learning and Systems*. 2024.

[19] Zuxin Liu et al. "APIGen: Automated Pipeline for Generating Verifiable and Diverse Function-Calling Datasets". In: *arXiv preprint arXiv:2406.18518* (2024).

[20] Meta AI. *Llama 3.2: Revolutionizing Edge AI and Vision with Open, Customizable Models*. 2024. URL: https://ai.meta.com/blog/llama-3-2-connected-devices-and-vision-models/ (visited on 06/05/2026).

[21] OpenAI. "GPT-4 Technical Report". In: *arXiv preprint arXiv:2303.08774* (2023).

[22] Shishir G Patil et al. “Gorilla: Large Language Model Connected with Massive APIs”. In: *arXiv preprint arXiv:2305.15334* (2023).
[23] Yujia Qin et al. “ToolLLM: Facilitating Large Language Models to Master 16000+ Real-world APIs”. In: *arXiv preprint arXiv:2307.16789* (2024).
[24] Qwen Team. “Qwen2.5 Technical Report”. In: *arXiv preprint arXiv:2412.15115* (2024).
[25] Raghav Sharma and Manan Mehta. “Small Language Models for Agentic Systems: A Survey of Architectures, Capabilities, and Deployment Trade-offs”. In: *arXiv preprint arXiv:2510.03847* (2025).
[26] Charlie Snell et al. “Scaling LLM Test-Time Compute Optimally can be More Effective than Scaling Model Parameters”. In: *arXiv preprint arXiv:2408.03314* (2024).
[27] Zhi Rui Tam et al. “Let Me Speak Freely? A Study on the Impact of Format Restrictions on Performance of Large Language Models”. In: *arXiv preprint arXiv:2408.02442* (2024).
[28] Ashish Vaswani et al. “Attention is All You Need”. In: *Advances in Neural Information Processing Systems*. Vol. 30. 2017.
[29] Thomas Wang et al. “What Language Model Architecture and Pretraining Objective Work Best for Zero-Shot Generalization?” In: *arXiv preprint arXiv:2204.05832* (2022).
[30] Jason Wei et al. “Chain-of-Thought Prompting Elicits Reasoning in Large Language Models”. In: *Advances in Neural Information Processing Systems*. Vol. 35. 2022.
[31] Brandon T Willard and Rémi Louf. “Efficient Guided Generation for Large Language Models”. In: *arXiv preprint arXiv:2307.09702* (2023).
[32] Fanjia Yan et al. *Berkeley Function Calling Leaderboard*. 2024. URL: https://gorilla.cs.berkeley.edu/leaderboard.html (visited on 06/04/2026).
[33] Shunyu Yao et al. “τ-bench: A Benchmark for Tool-Agent-User Interaction in Real-World Domains”. In: *arXiv preprint arXiv:2406.12045* (2024).
[34] Shunyu Yao et al. “ReAct: Synergizing Reasoning and Acting in Language Models”. In: *arXiv preprint arXiv:2210.03629* (2023).
[35] Jianguo Zhang et al. “xLAM: A Family of Large Action Models to Empower AI Agent Systems”. In: *arXiv preprint arXiv:2409.03215* (2024).

A Generative AI Disclosure

In accordance with DTU's guidelines on the use of generative AI tools in academic work, the following AI-assisted tools were used during the preparation of this thesis.

Claude Code (Anthropic) was used as a coding assistant during the implementation of the evaluation pipeline, HPC job scripts, and data processing scripts. Generated code was reviewed, tested, and modified before use. No generated code was submitted without verification against expected outputs.

Claude (Anthropic, claude.ai) was used as a writing assistant to produce draft text for thesis sections. **Codex in ChatGPT (OpenAI)** was also used as a writing assistant for revising thesis prose, improving structure, and clarifying explanations. All drafted or revised text was reviewed, modified, and rewritten in the author's own voice before inclusion. The author is responsible for all factual claims, experimental results, and interpretations presented in this thesis.

No AI tool was used to generate experimental data, benchmark results, or citations. All numerical results reported in this thesis were produced by running experiments on the DTU HPC cluster and collected by the author.

AI tools are not listed as authors or co-authors of this work, in accordance with the Vancouver Convention as adopted by DTU [13].

B Hyperparameters and Training Details

Table B.1 lists the hyperparameters used for LoRA fine-tuning in all training configurations.

Table B.1: LoRA fine-tuning hyperparameters for all training configurations.

Parameter	Value
Base model	`Qwen/Qwen2.5-7B-Instruct`
Dataset	`Salesforce/xlam-function-calling-60k`
Training examples	54,000
Validation examples	6,000
LoRA rank	16
LoRA alpha	32
Target modules	`q_proj, v_proj`
Dropout	0.05
Epochs	2
Batch size	4
Gradient accumulation steps	4
Learning rate	2×10^{-4}
LR scheduler	cosine
Dtype	bfloat16
Optimizer	AdamW
Max sequence length	2048

Fig. B.1 shows the training loss for the two LoRA adapters discussed in Section 4.4.1.

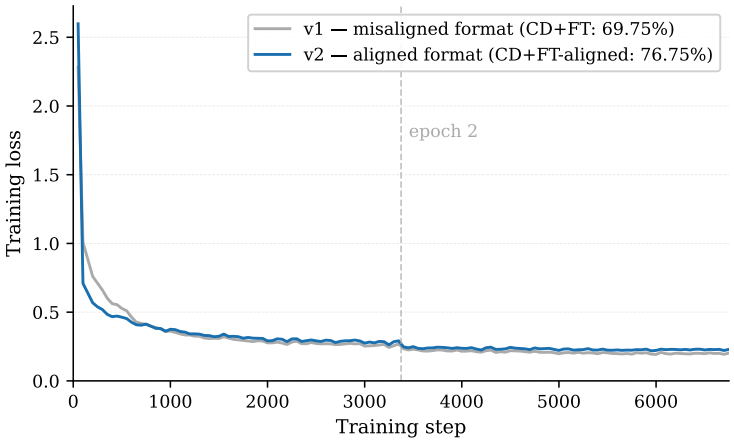

Figure B.1: Training loss for the misaligned (v1) and format-aligned (v2) LoRA adapters over 6,750 steps (2 epochs on 54,000 xlam examples). Both runs converge to comparable loss values; the 7.0 pp accuracy difference between CD+FT and CD+FT-aligned is caused by the training output format, not by training quality.

C Full Benchmark Results

Table C.1 reports the complete per-configuration results on BFCL v4 `simple_python` (400 test cases).

Table C.1: Full BFCL v4 `simple_python` AST accuracy results across all configurations evaluated in this thesis. Config B reports the strict whole-completion-parse floor; the no-guided rows (RAG-ng, PE-ng, ITC-ng, FT-only, FT-aligned-ng) use the lenient first-complete-object parser, matching the isolation arm in Section 4.2.7 (Tables 4.5 and 4.6). The two parse criteria are defined in Section 3.4.

Config	Description	Correct	Accuracy (↑)	Delta vs CD
B	No constrained decoding	6/400	1.50%	−71.25 pp
B-template	Native template, free generation	384/400	96.00%	+23.25 pp
PE	CD + few-shot (3 examples)	281/400	70.25%	−2.50 pp
CD	Constrained decoding	291/400	72.75%	baseline
CD+Q	CD + AWQ INT4	289/400	72.25%	−0.50 pp
CD+Q+ITC	CD+Q + chain-of-thought	262/400	65.50%	−7.25 pp
CD+Q+RAG	CD+Q + FAISS top-5	191/400	47.75%	−25.00 pp
RAG-ng	Retrieval, no CD	208/400	52.00%	−20.75 pp
PE-ng	Few-shot, no CD	236/400	59.00%	−13.75 pp
ITC-ng	Chain-of-thought, no CD	225/400	56.25%	−16.50 pp
FT-only	LoRA merged, no CD	212/400	53.00%	−19.75 pp
CD+FT	CD + LoRA merged (v1)	279/400	69.75%	−3.00 pp
FT-aligned-ng	Format-aligned LoRA, no CD	164/400	41.00%	−31.75 pp
CD+FT-aligned	CD + format-aligned LoRA	307/400	76.75%	+4.00 pp
CD+Q+FT-aligned	CD + AWQ INT4 + format-aligned LoRA	297/400	74.25%	+1.50 pp
CD+schema	**CD + schema-enriched prompt**	**356/400**	**89.00%**	**+16.25 pp**

D Updated Project Plan: Deviations from the Original Plan

This appendix updates the project plan submitted at the start of the project, as required for hand-in. It records the parts of the project that changed significantly from the original plan; the original motivation, research question, and time plan are otherwise unchanged.

D.1 Scope and framing

The original plan positioned constrained decoding (CD) as the core contribution, supported by a set of complementary efficiency techniques (quantization, LoRA fine-tuning, retrieval-augmented generation, and possibly knowledge distillation). The final thesis keeps CD as the core, but the framing shifted to expressing each technique as a candidate for *expanding the boundary of a cascade LLM architecture*, i.e. how far a small model can be pushed before a request must be escalated to a larger model.

D.2 Changes to the experimental program

The most significant deviation concerns the role of the complementary techniques. The original plan treated them as additive improvements on top of CD. The experiments showed the opposite for several of them:

- Retrieval-augmented generation reduced accuracy substantially (-25 pp relative to plain CD) rather than improving it.
- In-context chain-of-thought (ITC) also reduced accuracy (-7.25 pp).
- AWQ INT4 quantization was approximately neutral (-0.50 pp), confirming it as a low-cost compression option rather than an accuracy lever.
- LoRA fine-tuning helped only modestly and only in combination with CD (format-aligned LoRA: $+4.00$ pp), and collapsed to near zero accuracy without guided decoding.

Knowledge distillation, listed in the original plan as an optional alternative, was not pursued; it was de-scoped to keep the analysis focused on the techniques above, as anticipated by the risk-mitigation note in the original plan.

The strongest configuration in the final thesis, `CD+schema` (a schema-enriched argument-extraction prompt, $+16.25$ pp over plain CD), was not part of the original plan. It emerged from analysing the failure modes observed during the constrained-decoding experiments and became a central result.

D.3 Time plan

E Self-Evaluation of the Project Process

Technical
University of
Denmark

Richard Petersens Plads, Building 324
2800 Kgs. Lyngby
Tlf. 4525 1700

www.compute.dtu.dk